

LIBRO DE TEORÍA · EDICIÓN DE ESTUDIO

AWS Certified GenAI Developer

Teoría completa de Amazon Bedrock, RAG, Agentes, Guardrails, Monitorización, Rendimiento y Gobernanza — construida a partir del análisis de 97 escenarios reales de examen y la documentación oficial de AWS.

- 9 módulos · 8 diagramas de arquitectura
- Casos de estudio basados en escenarios reales de examen
- Referencias verificadas contra docs.aws.amazon.com

Índice

MÓDULO 0: FUNDAMENTOS DE IA GENERATIVA Y MAPA DE COMPETENCIAS	4
¿Qué es la IA Generativa y qué es un Foundation Model?	4
Vocabulario que aparece en casi todas las preguntas	5
Por qué Amazon Bedrock	5
Los siete bloques temáticos del examen	6
Cómo están construidas las preguntas y cómo estudiarlas	6
1.1 Qué es Amazon Bedrock	7
1.2 Converse/ConverseStream frente a InvokeModel/InvokeModelWithResponseStream	8
1.3 Modos de inferencia en profundidad	9
1.4 Personalización de modelos: fine-tuning y continued pre-training	10
Fuentes	13
2.1 Qué resuelve Bedrock Knowledge Bases y qué no	14
2.2 Elegir el almacén vectorial correcto	14
2.3 Estrategias de chunking y un matiz que el examen explota	15
2.4 Metadata filtering, búsqueda híbrida y reranking: tres capas de precisión distintas	16
2.5 Query Decomposition: cuando la pregunta del usuario es el problema	17
2.6 Citas, explicabilidad y por qué RetrieveAndGenerate gana casi siempre	17
2.7 Reducir alucinaciones: RAG no es magia por sí solo	18
2.8 Patrones de arquitectura multi-cuenta, multi-región y de ingesta	18
Fuentes	19
3.1 El patrón ReAct y el ciclo de vida de un Bedrock Agent	20
3.2 Anatomía de un Agent: FM, Instruction, Action Groups, Knowledge Bases	21
3.3 Ejecución de acciones: AWS Lambda vs Return of Control (RoC)	22
3.4 Memoria: de la sesión con estado a Amazon Bedrock AgentCore	22
3.5 Multi-Agent Collaboration: arquitectura jerárquica y aislamiento por dominio	23
3.6 Amazon Bedrock Flows: orquestación visual y determinista	23
3.7 Cuando NO usar un Agente: orquestación determinista con AWS Step Functions	24
3.8 Configuración dinámica sin redespigie: AWS AppConfig	25
3.9 Model Context Protocol (MCP) en producción sobre AWS	26
3.10 Elegir la herramienta correcta: Agent, Flow o Step Functions	27
Fuentes	27
4.1 Guardrails como capa de seguridad independiente del modelo	28
4.2 Denied Topics: bloqueo semántico de intenciones	29
4.3 Content Filters: las seis categorías de daño y sus niveles de confianza	29
4.4 Sensitive Information Filters (PII): Mask vs Block, y el matiz de residencia de datos	30
4.5 Word Filters y listas de bloqueo por coincidencia exacta	31
4.6 Contextual Grounding Check: la defensa nativa contra alucinaciones en RAG	31
4.7 Aplicación práctica: versionado, ApplyGuardrail y observabilidad	32
Fuentes	33

5.1 Model Evaluation en Amazon Bedrock: automática vs. humana	34
5.2 Framework RAGAS y evaluación LLM-as-a-Judge en producción	35
5.3 Amazon SageMaker Clarify: sesgo, equidad y deriva	36
5.4 Bedrock Trace Events: depurando el razonamiento de un agente	37
5.5 Model Invocation Logging y observabilidad de tokens	37
5.6 Contextual Grounding Check: el detector de alucinaciones en tiempo real	38
Fuentes	40
6.1 Latencia percibida: streaming de respuestas	41
6.2 Prompt Caching en profundidad	43
6.3 Resiliencia ante Throttling: reintentos, modo adaptativo y circuit breaker	44
6.4 Coste: eligiendo el modo de inferencia correcto	46
Fuentes	46
7.1 Control centralizado con Organizations y Service Control Policies	47
7.2 Aislamiento de red con AWS PrivateLink	48
7.3 Gobernanza de datos: AWS Lake Formation frente a S3 y Glue	48
7.4 Validación preventiva de infraestructura: AWS CloudFormation Guard	49
7.5 Gobernanza de Amazon Q Business y Amazon Q Developer	50
7.6 Gestión del ciclo de vida de prompts: Amazon Bedrock Prompt Management	51
7.7 Auditoría, cifrado y visibilidad operativa	51
Fuentes	52
Tabla de Decisión Rápida: “¿Qué Servicio / Técnica Elegir?”	53
Los 10 “Gotchas” y Trampas Frecuentes del Examen	54

MÓDULO 0: FUNDAMENTOS DE IA GENERATIVA Y MAPA DE COMPETENCIAS

¿Qué es la IA Generativa y qué es un Foundation Model?

La IA Generativa es la rama del *machine learning* dedicada a modelos capaces de **producir contenido nuevo** —texto, código, imágenes, audio— a partir de un contexto de entrada, en lugar de limitarse a clasificar o predecir un valor sobre datos ya existentes. El motor detrás de casi toda aplicación de IA Generativa moderna es el **Foundation Model (FM)**: un modelo de aprendizaje profundo, normalmente basado en la arquitectura *transformer*, entrenado de forma autosupervisada sobre volúmenes masivos de datos (texto de internet, código, libros, imágenes) durante una fase de **preentrenamiento** que cuesta millones de dólares en cómputo. El resultado es un modelo con un conocimiento general amplio del lenguaje, el razonamiento y el mundo, que después puede **adaptarse** a tareas concretas sin tener que repetir ese entrenamiento masivo desde cero.

Esa capacidad de adaptación es exactamente lo que Amazon Bedrock explota, y entenderla como un *espectro* de menor a mayor coste/esfuerzo es la primera idea que hay que interiorizar para el examen, porque una parte enorme de las preguntas gira alrededor de “¿cuál de estas cuatro técnicas es la más barata/rápida/sencilla para lograr X?”:

1. **Prompt Engineering** (coste casi nulo, inmediato): se reformula la instrucción, se añaden ejemplos *few-shot* o se estructura mejor el contexto, sin tocar el modelo ni añadir datos externos en tiempo de ejecución. Es siempre la primera palanca a probar y, en el examen, casi nunca es la respuesta “correcta” cuando el escenario exige datos propietarios o actualizados, porque el modelo solo “sabe” lo que aprendió en el preentrenamiento.
2. **RAG (Retrieval-Augmented Generation)** (coste bajo-medio, sin reentrenar): en el momento de la consulta se recuperan fragmentos relevantes de una base de conocimiento propia y se inyectan en el prompt como contexto adicional. El modelo no cambia; lo que cambia es la información que ve. Es la técnica preferida cuando el requisito es “responder con datos actualizados o privados de la empresa” — se cubre en profundidad en el Módulo 2.
3. **Fine-Tuning / Personalización de modelos** (coste alto, requiere datos etiquetados y tiempo de entrenamiento): se ajustan los pesos del modelo con ejemplos propios para especializar su *estilo, formato de salida o dominio*. En Bedrock, un modelo personalizado (*Custom Model*) exige después **Provisioned Throughput** para poder invocarse (Módulo 1) — un matiz que el examen pregunta explícitamente.
4. **Preentrenamiento desde cero** (coste extremo, prácticamente nunca es la respuesta correcta en un examen orientado a “desarrollador”): reservado a proveedores del modelo, no a equipos de aplicación.

La regla general de examen es: si el enunciado no exige cambiar el comportamiento fundamental del modelo ni su conocimiento factual privado, la solución de menor esfuerzo es prompt engineering; si exige datos propios o actualizados, es RAG; solo si exige cambiar el *comportamiento, tono o formato* de forma consistente y a gran escala se justifica el fine-tuning.

Vocabulario que aparece en casi todas las preguntas

Antes de entrar en Bedrock conviene fijar un pequeño vocabulario que el examen usa sin volver a explicar:

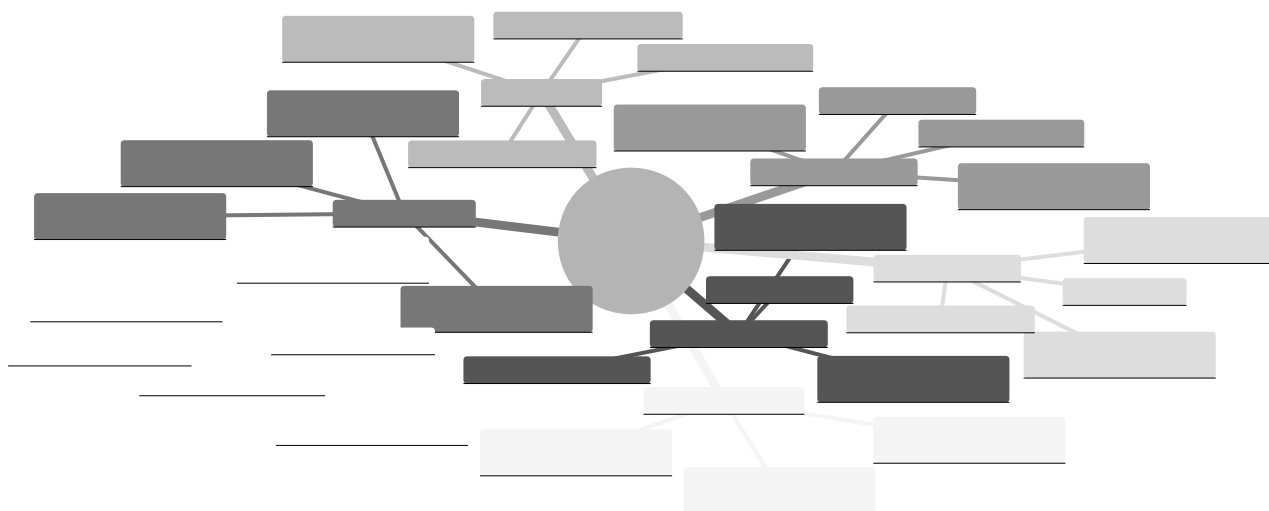
- **Token:** unidad mínima de texto que procesa el modelo (aproximadamente $\frac{3}{4}$ de palabra en inglés). El coste de Bedrock y los límites de las APIs se miden en tokens de entrada y de salida, no en caracteres ni en palabras.
- **Ventana de contexto (*context window*):** número máximo de tokens (entrada + salida) que un modelo puede “ver” en una sola llamada. Es el límite que obliga a diseñar estrategias de *chunking* en RAG y de *prompt caching* en aplicaciones de alto volumen.
- **Embeddings:** representación numérica (un vector de cientos o miles de dimensiones) del significado semántico de un texto. Dos fragmentos con significado similar producen vectores cercanos en ese espacio; esta propiedad es la base matemática de toda búsqueda semántica y, por tanto, de RAG.
- **Parámetros de inferencia — Temperature, Top-P, Top-K:** controlan la aleatoriedad de la generación. *Temperature* alta favorece respuestas más creativas/variadas; baja favorece respuestas deterministas y repetibles. *Top-K* limita el muestreo a los K tokens más probables en cada paso; *Top-P* (*nucleus sampling*) limita el muestreo al conjunto de tokens cuya probabilidad acumulada supera P. El examen suele usar estos parámetros como distractor: bajar la temperatura a 0 reduce la creatividad, pero **no** garantiza que el modelo no alucine ni sustituye a un Guardrail de *Contextual Grounding* (ver Módulo 4).
- **Latencia vs. Throughput:** la latencia es el tiempo hasta obtener una respuesta (o el primer token, *Time To First Token*); el throughput es el volumen de solicitudes o tokens que el sistema puede sostener por unidad de tiempo. Muchas preguntas de rendimiento (Módulo 6) exigen distinguir cuál de los dos es realmente el cuello de botella del escenario antes de elegir la solución.
- **Coste por token vs. coste por hora:** los modelos On-Demand se facturan por token procesado; el Provisioned Throughput se factura por Model Unit y por hora reservada, independientemente del volumen real de tráfico. Confundir ambos modelos de precio es una trampa recurrente.

Por qué Amazon Bedrock

Antes de Bedrock, construir una aplicación de IA Generativa exigía elegir un proveedor de modelo, gestionar sus SDKs específicos, aprovisionar infraestructura de GPU (propia o de terceros) y resolver por cuenta propia la seguridad, el registro de auditoría y el escalado. Amazon Bedrock es, según la documentación oficial de AWS, “un servicio completamente gestionado que proporciona acceso seguro y de nivel empresarial a foundation models de alto rendimiento de las principales compañías de IA, permitiendo construir y escalar aplicaciones de IA Generativa” sin gestionar servidores ni clústeres de entrenamiento (docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html). Bedrock agrega en una sola superficie de API y de facturación más de un centenar de modelos de proveedores como Amazon (familia Nova), Anthropic (Claude), Meta (Llama), Mistral AI, DeepSeek, Cohere y AI21 Labs, entre otros, y añade sobre ellos las capas transversales que el resto de este libro desarrolla: Knowledge Bases para RAG, Agents y Flows para orquestación, Guardrails para seguridad de contenido, y evaluación/observabilidad integradas con CloudWatch y SageMaker Clarify.

Esa naturaleza “serverless y multi-proveedor” es la clave de una de las reglas de examen más repetidas: cuando un escenario necesita poder **cambiar de modelo o soportar varios proveedores con el mínimo esfuerzo de desarrollo**, la respuesta casi siempre pasa por la capa de abstracción que Bedrock ofrece para eso —

típicamente la API `Converse` (Módulo 1) — en lugar de construir integraciones a medida contra el SDK propio de cada proveedor.



Mapa de competencias evaluadas en el examen

Los siete bloques temáticos del examen

El banco de 97 escenarios reales sobre el que se ha construido este libro se agrupa, de forma casi perfectamente equilibrada, en siete grandes bloques de competencia, que son exactamente los siete módulos que siguen:

1. **Amazon Bedrock Core** (Módulo 1): las APIs de inferencia (`Converse` vs `InvokeModel`), los modos de consumo (On-Demand, Provisioned Throughput, Batch) y la inferencia multi-región.
2. **RAG y Knowledge Bases** (Módulo 2): estrategias de *chunking*, bases de datos vectoriales, búsqueda híbrida y *reranking*.
3. **Agentes y Orquestación** (Módulo 3): el patrón ReAct, *Action Groups*, *Return of Control*, memoria de agente y Bedrock Flows — es, junto con RAG, el bloque con más peso en el examen.
4. **Guardrails y Seguridad de Contenido** (Módulo 4): temas denegados, filtros de contenido, PII y *contextual grounding*.
5. **Monitorización y Evaluación** (Módulo 5): evaluación automática vs. humana, RAGAS, trazas de Bedrock y SageMaker Clarify.
6. **Rendimiento y Coste** (Módulo 6): streaming, *prompt caching*, *throttling* y desacoplamiento asíncrono.
7. **Seguridad y Gobernanza** (Módulo 7): SCPs, PrivateLink, Lake Formation y validación de IaC en CI/CD.

Cómo están construidas las preguntas y cómo estudiarlas

Casi ninguna pregunta del examen pide una definición aislada (“¿qué es X?”). El formato dominante es el **escenario largo**: una empresa ficticia describe su caso de uso, sus restricciones técnicas y, sobre todo, **un criterio de decisión explícito** —“con el MENOR esfuerzo de desarrollo posible”, “sin coste operativo adicional”, “cumpliendo con la residencia de datos en la UE”, “sin exponer las credenciales corporativas fuera de AWS”— y pide elegir, entre cuatro opciones, la única que satisface ese criterio, no solo la que “funcionaría”.

Esto tiene una consecuencia práctica para estudiar: **casi todas las opciones incorrectas son técnicamente plausibles**. Suelen ser servicios de AWS reales, usados de forma razonable, que resuelven un problema parecido pero no exactamente el que pide el criterio oculto del enunciado. Por eso cada capítulo de este libro no se limita a explicar “qué hace” cada servicio, sino que dedica secciones explícitas a los **matices que separan a la opción correcta de sus distractores más plausibles** — precisamente el patrón de razonamiento (“por qué A, B y D fallan y por qué C es correcta”) que reproducen las explicaciones oficiales del banco de preguntas en el que se basa este libro. Al estudiar, la pregunta que hay que hacerse siempre no es “¿qué servicio de AWS podría resolver esto?” sino “¿qué palabra del enunciado descarta a los otros tres?”.

Con esa lente en mente, el Módulo 1 empieza donde toda aplicación de IA Generativa en AWS empieza: las APIs de inferencia de Amazon Bedrock. ## MÓDULO 1: AMAZON BEDROCK — FUNDAMENTOS Y APIS DE INFERENCIA

1.1 Qué es Amazon Bedrock

Amazon Bedrock es el servicio de AWS que expone, mediante una única API administrada y sin servidor (serverless), el acceso a un catálogo de Foundation Models (FMs) de distintos proveedores: Anthropic (familia Claude), Meta (Llama), Mistral AI, Amazon (familia Nova y el histórico Titan), Cohere, AI21 Labs, DeepSeek, OpenAI (mediante los modelos gpt-oss y una API de compatibilidad OpenAI), entre otros. La palabra clave que el examen espera que el candidato entienda en profundidad es “serverless”: Bedrock no obliga a aprovisionar, parchear ni escalar manualmente instancias de cómputo (GPU o de otro tipo) para poder invocar un modelo. AWS gestiona la infraestructura subyacente, y el desarrollador solo se preocupa por el contrato de la API — peticiones y respuestas HTTP/JSON firmadas con SigV4 e IAM— y por el modo de facturación que elige (on-demand, aprovisionado o por lotes, como se detalla más adelante).

Esta arquitectura tiene dos consecuencias directas que aparecen constantemente en las preguntas de examen. La primera es que el control de acceso a cada modelo se gestiona con políticas de IAM sobre acciones como `bedrock:InvokeModel`, `bedrock:InvokeModelWithResponseStream`, `bedrock:Converse` o `bedrock:ConverseStream`, en lugar de credenciales de aplicación separadas por proveedor de modelo: da igual si el modelo detrás es Claude o Nova, la autenticación siempre pasa por el mismo mecanismo de roles de IAM del propio servicio Bedrock. La segunda es que, al no existir instancias dedicadas al modelo por defecto, Bedrock necesita ofrecer varios “modos de inferencia” —on-demand, aprovisionado y por lotes— para cubrir perfiles de carga muy distintos, y el examen evalúa sistemáticamente si el candidato sabe emparejar cada escenario de negocio con el modo correcto.

Por encima de esos modos de inferencia, Bedrock añade una capa de API unificada (`Converse` / `ConverseStream`) que resuelve un problema práctico real: cada proveedor de modelos definió originalmente su propio formato de petición y de respuesta JSON, lo que complicaba escribir código portable entre modelos. Entender por qué existe esa capa —y cuándo conviene evitarla— es el segundo bloque teórico de este módulo.

1.2 Converse/ConverseStream frente a InvokeModel/InvokeModelWithResponseStream

Bedrock ofrece dos familias de operaciones de inferencia, ambas expuestas únicamente en el endpoint de plano de datos `bedrock-runtime` (no en el plano de control `bedrock`, que se usa para tareas administrativas como crear Provisioned Throughput o gestionar modelos personalizados).

`InvokeModel` e `InvokeModelWithResponseStream` son las operaciones de bajo nivel, prácticamente un paso directo hacia el modelo subyacente: el cuerpo de la petición (`body`) es un JSON cuya estructura depende por completo del proveedor del modelo. Invocar un modelo de Anthropic requiere un esquema de mensajes con campos como `anthropic_version`, `max_tokens` y un array `messages`; invocar un modelo de la familia Titan de Amazon requiere un esquema distinto con `inputText` y `textGenerationConfig`. Esto significa que cualquier aplicación que quiera admitir varios modelos, o migrar de un modelo a otro, debe mantener lógica de serialización/deserialización distinta para cada proveedor. La ventaja de esta familia de operaciones es el acceso íntegro a todos los parámetros específicos de cada modelo, incluidos los más nuevos o menos habituales que todavía no se hayan mapeado al formato común.

`Converse` y `ConverseStream` (esta última siendo la variante con streaming de tokens en tiempo real) nacen precisamente para eliminar esa fricción. Definen un esquema de solicitud y de respuesta idéntico para todos los modelos de Bedrock que admiten conversación: un array `messages` donde cada mensaje tiene un `role` (`user` o `assistant`, nunca `system` dentro del array: las instrucciones de sistema se pasan aparte, en un campo `system` de nivel superior de la petición), un bloque `content` que puede combinar texto, imágenes o documentos, un objeto `inferenceConfig` para parámetros comunes como `maxTokens`, `temperature` o `topP`, y —crucialmente— un mecanismo estandarizado de tool use / function calling mediante los campos `toolConfig` (para declarar las herramientas disponibles con su esquema JSON de entrada) y `toolUse` / `toolResult` (para que el modelo solicite invocar una herramienta y la aplicación le devuelva el resultado en el turno siguiente). Gracias a esa estandarización, el mismo código de orquestación de un agente puede intercambiar Claude por Nova o por Llama sin reescribir la capa de parseo, siempre que el caso de uso no dependa de un parámetro exclusivo de un proveedor concreto; y si lo necesita, `Converse` permite igualmente inyectar parámetros adicionales específicos del modelo a través de `additionalModelRequestFields`, sin perder la compatibilidad del resto del esquema.

El razonamiento que el examen premia no es memorizar “usa `Converse` por defecto”, sino entender el porqué: cuando el enunciado describe una aplicación que debe (a) funcionar igual en varios lenguajes de programación o SDKs, (b) sobrevivir a un cambio o a una comparación entre FMs, o (c) mantener contexto conversacional multi-turno de forma sencilla —reenviando el propio array de `messages` con los turnos anteriores, sin infraestructura de estado dedicada—, la respuesta correcta casi siempre pasa por `Converse` / `ConverseStream`. Por el contrario, cuando el enunciado exige un parámetro de inferencia exclusivo de un proveedor que aún no está expuesto en el esquema común, o cuando ya existe una integración heredada fuertemente acoplada al formato nativo de un modelo concreto, `InvokeModel` / `InvokeModelWithResponseStream` sigue siendo la elección correcta. Ambas familias comparten el mismo modelo de autenticación por IAM y ambas pueden apuntar tanto a un modelo base on-demand como a un ARN de Provisioned Throughput o a un perfil de inferencia entre regiones como `modelId`, así que la elección entre una y otra depende del formato de payload que necesita la aplicación, no del modo de facturación o de capacidad que se esté usando por debajo.

Conviene distinguir también estas dos operaciones de los dos métodos de invocación orientados a RAG con Knowledge Bases: `RetrieveAndGenerate`, que en una sola llamada consulta la base de conocimiento vectorial

y genera la respuesta final con citas a las fuentes recuperadas, y `Retrieve`, que solo devuelve los fragmentos relevantes sin invocar ningún FM, dejando la generación en manos de la aplicación (típicamente porque esta necesita combinar esos fragmentos con su propia lógica de prompting, orquestación de agentes o post-procesado antes de generar la respuesta).

1.3 Modos de inferencia en profundidad

On-Demand. Es el modo de facturación por defecto: se paga por token de entrada y de salida consumido, sin ningún compromiso de capacidad ni de tiempo. La contrapartida es que la capacidad on-demand es compartida entre todos los clientes de una región y está sujeta a cuotas de solicitudes por minuto (RPM) y tokens por minuto (TPM) específicas de cada modelo; superarlas produce respuestas de throttling (error `ThrottlingException`) que la aplicación debe absorber, típicamente con reintentos con backoff exponencial. La documentación reciente de Bedrock matiza este modo distinguiendo niveles de servicio dentro del propio on-demand —Flex (el más económico, orientado a cargas tolerantes a demoras y con mayor riesgo de throttling, sin SLA), Standard (el equilibrio habitual de producción) y Priority (mayor coste por token a cambio de una asignación de throughput preferente y menor riesgo de throttling)— pero en cualquiera de sus variantes sigue siendo capacidad compartida y elástica, nunca capacidad reservada en exclusiva para la cuenta.

Provisioned Throughput. Resuelve exactamente el problema que el on-demand no puede garantizar: capacidad de inferencia dedicada y predecible, medida en Model Units (MUs). Cada MU representa una cantidad fija de capacidad de procesamiento (tokens de entrada y de salida por unidad de tiempo) que depende del modelo concreto; al comprar Provisioned Throughput se especifica el número de MUs necesarias y, opcionalmente, un `commitmentDuration`. Según la API `CreateProvisionedModelThroughput`, ese campo acepta únicamente los valores `OneMonth` o `SixMonths`; si se omite, se obtiene una Provisioned Throughput sin compromiso, facturada por hora y cancelable en cualquier momento, aunque a un coste por MU-hora superior al de los compromisos más largos (la duración más larga se paga con un descuento por hora a cambio de una permanencia mínima). Un matiz que el examen explota deliberadamente es que comprar Provisioned Throughput no basta por sí solo: hay que dirigir explícitamente las llamadas de inferencia (`InvokeModel`, `InvokeModelWithResponseStream`, `Converse` o `ConverseStream`) al ARN del modelo aprovisionado —el campo `provisionedModelArn` que devuelve `CreateProvisionedModelThroughput` — en el parámetro `modelId` de la petición; si el código sigue usando el identificador del modelo base on-demand, todo el tráfico continúa yendo a la capacidad compartida, la capacidad reservada aparece como no utilizada en las métricas de CloudWatch, y el throttling en el tráfico on-demand no desaparece, por mucho que se incrementen las MUs compradas. Otro matiz igualmente importante: Provisioned Throughput garantiza capacidad y ausencia de throttling dentro del volumen contratado, pero no es, por sí misma, una técnica de reducción de latencia por solicitud; para eso existe una capacidad distinta, Latency Optimized Inference, que se activa fijando el parámetro `latency` a `optimized` dentro de `performanceConfig` en la propia solicitud de inferencia, sin necesidad de reservar ni pagar capacidad fija, y con la particularidad de que si se agota la cuota de aceleración la petición simplemente se procesa a la latencia estándar en lugar de fallar. Por último, Provisioned Throughput es obligatorio —no opcional— para poder invocar cualquier modelo personalizado (fine-tuned o de continued pre-training) en producción, como se explica en la sección siguiente.

Batch Inference. Es el modo pensado para procesar grandes volúmenes de datos sin restricciones de latencia por solicitud individual: se define un trabajo que lee las peticiones desde un bucket de Amazon S3 (en un formato JSONL con un registro por línea) y escribe los resultados en otro bucket o prefijo de S3, procesándolos

todos de forma asíncrona. Según la documentación oficial de capacidad y optimización de costes de Bedrock, este modo ofrece un ahorro de aproximadamente el 50% frente al precio on-demand por token, dentro de una ventana de procesamiento de hasta 24 horas y con límites de tamaño de trabajo (hasta 10.000 registros y un archivo de entrada de hasta 200 MB por trabajo, cifras que pueden variar por región y modelo). Es la opción correcta siempre que el enunciado describa generación de datos de entrenamiento, backlogs de moderación de contenido, generación masiva de informes o enriquecimiento de datos: cualquier caso donde no exista un usuario esperando la respuesta en tiempo real.

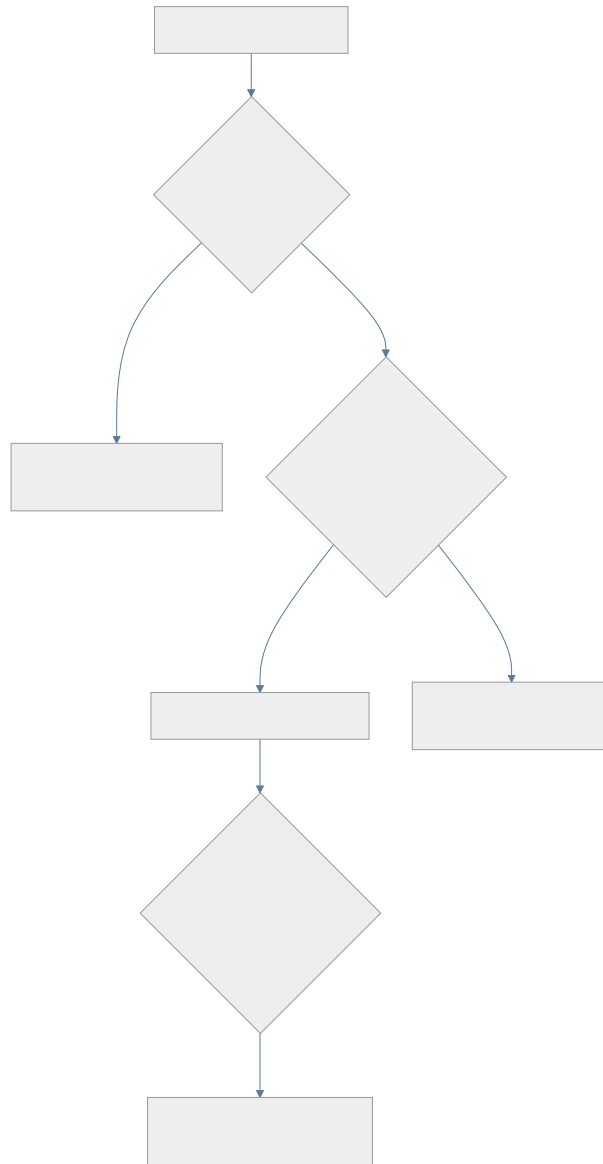
Cross-Region Inference (perfiles de inferencia). No es un modo de facturación distinto —de hecho sigue tarifándose a precio on-demand, sin coste adicional de enrutamiento ni de transferencia de datos— sino un mecanismo de enrutamiento de solicitudes de inferencia. Un perfil de inferencia con prefijo geográfico (`us.`, `eu.`, `apac.`, entre otros) permite que Bedrock distribuya automáticamente las solicitudes entre varias regiones AWS de destino dentro de esa misma área geográfica, aumentando la capacidad efectiva disponible y mejorando la resiliencia frente a picos de tráfico o interrupciones puntuales de una región, sin que el desarrollador tenga que implementar lógica de reintento o failover entre regiones. Es fundamental no confundir esto con el movimiento de datos de entrenamiento: cross-Region inference nunca traslada los pesos del modelo ni afecta a cómo se entrenó; únicamente puede mover, de forma cifrada, el contenido de la solicitud de inferencia (el prompt de entrada y la respuesta generada) hacia otra región de la misma área geográfica para procesarla, y por defecto los datos permanecen almacenados solo en la región de origen. Por esta razón, en escenarios de residencia de datos estrictos por país (no solo por continente), un perfil geográfico “eu.” puede no ser suficiente si el requisito exige permanecer en una única región concreta, y conviene revisar qué regiones de destino incluye cada perfil antes de asumir que cumple un requisito de soberanía de datos. Cross-Region Inference sí es compatible con Provisioned Throughput, aunque puede requerir permisos de IAM adicionales sobre el modelo fundacional en todas las regiones de origen y de destino del perfil, no solo en la región donde se invoca.

1.4 Personalización de modelos: fine-tuning y continued pre-training

Bedrock permite personalizar un subconjunto de FMs mediante dos técnicas de model customization: el fine-tuning (ajuste fino supervisado con pares de entrada/salida etiquetados, típicamente para especializar el estilo de respuesta o el formato de salida a un dominio concreto) y el continued pre-training (entrenamiento adicional no supervisado sobre un corpus de texto propio del dominio, para que el modelo absorba vocabulario y conocimiento específico sin depender de etiquetas). En ambos casos, el proceso de entrenamiento se ejecuta como un trabajo asíncrono que lee los datos de entrenamiento desde Amazon S3 y produce un “modelo personalizado” (custom model) que queda registrado dentro de la cuenta de Bedrock, asociado siempre al modelo base del que deriva.

La razón teórica —no solo la regla memorística— por la que estos modelos personalizados exigen Provisioned Throughput para poder invocarlos es que un modelo con pesos ajustados deja de ser parte del pool de capacidad compartida on-demand que Bedrock gestiona de forma multiinquilino para los modelos base: sus pesos, distintos a los del modelo base estándar, necesitan una copia de inferencia dedicada dentro de la infraestructura de Bedrock para poder servirse, y esa copia dedicada es exactamente lo que se paga al comprar Model Units. Por eso, aunque para probar un modelo personalizado durante la fase de evaluación se puede usar Provisioned Throughput sin compromiso, cualquier uso sostenido en producción implica necesariamente ese coste de

capacidad dedicada, algo que conviene modelar en el análisis de costes desde el diseño del proyecto y no como un descubrimiento tardío tras el entrenamiento.



Árbol de decisión: cómo elegir el modo de inferencia adecuado

Caso de Estudio

Una empresa que procesa informes financieros necesita ejecutar el mismo flujo de análisis en dos entornos muy distintos: funciones Lambda en Python que atienden peticiones puntuales, y contenedores en Amazon EKS que procesan lotes de informes en JavaScript según una programación. El reto no es solo técnico sino de coherencia: el mismo modelo fundacional debe responder igual en ambos entornos, la autenticación debe ser uniforme y el contexto de una conversación multi-turno debe conservarse entre llamadas sucesivas. La tentación de usar `InvokeModel` directamente falla porque obliga a mantener lógica de formateo de payload distinta para cada SDK y complica tener una autenticación consistente entre Lambda y EKS. Añadir una base de datos externa (por ejemplo Amazon ElastiCache) para guardar el estado de la conversación tampoco es necesario: la propia Converse API ya expone un esquema de solicitud/respuesta idéntico en todos los SDK de AWS, así que basta con reenviar el array `messages` completo —incluyendo los turnos anteriores con sus roles `user` / `assistant`— en cada nueva llamada para mantener el contexto, y usar roles de IAM con permiso sobre `bedrock:InvokeModel` como mecanismo de autenticación común a ambos entornos. La lección de fondo es que la Converse API no solo simplifica el código: elimina por diseño la necesidad de infraestructura de estado dedicada para casos de multi-turno sencillos, porque el propio protocolo de mensajes ya es el estado.

Caso de Estudio

Un asistente de atención al cliente de una tienda online recibe un volumen de consultas donde la mayoría son preguntas sencillas sobre productos y una minoría son consultas complejas sobre política de devoluciones que exigen razonamiento más profundo. El objetivo de negocio es enrutar cada consulta al modelo más adecuado sin construir una arquitectura de clasificación propia. Construir un clasificador con un FM adicional y funciones Lambda para dirigir el tráfico funciona, pero implica diseñar, probar y mantener una arquitectura multietapa completa. Crear endpoints separados con reglas basadas en palabras clave tiene el mismo problema, y añadir Provisioned Throughput para el modelo grande suma además un compromiso de capacidad que puede no estar justificado. La solución de mínimo esfuerzo es el intelligent prompt routing nativo de Bedrock: un único endpoint sin servidor que predice, para cada solicitud entrante, qué modelo de una misma familia ofrecerá el mejor equilibrio entre calidad de respuesta y coste, y dirige automáticamente el tráfico sencillo hacia un modelo pequeño y económico y el tráfico complejo hacia un modelo más grande y capaz, ya sea usando un router predeterminado de AWS o configurando uno propio. La idea clave es que “mínimo esfuerzo de implementación” en el enunciado de un examen suele ser la pista textual que apunta hacia una capacidad gestionada de la plataforma, no hacia una arquitectura de orquestación construida a medida, aunque esa arquitectura a medida sea perfectamente viable técnicamente.

Caso de Estudio

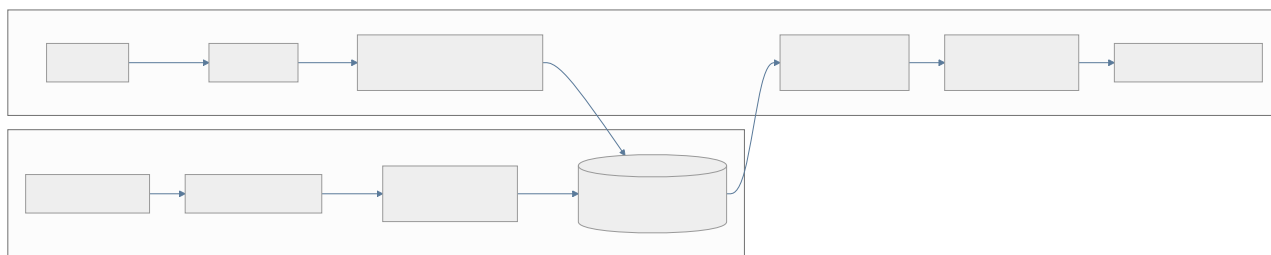
Una compañía de servicios financieros compra Provisioned Throughput para sostener un volumen constante de solicitudes durante el horario laboral, pero observa en CloudWatch que la capacidad reservada permanece sin usar mientras las solicitudes on-demand siguen sufriendo throttling en las horas de máxima carga. La causa no está en la cantidad de Model Units contratadas ni se soluciona añadiendo lógica de reintento con backoff: el código de la aplicación sigue invocando el modelo mediante su identificador de modelo base on-demand, de modo que ninguna solicitud llega nunca a la capacidad dedicada, sin importar cuánto se incremente. La corrección consiste en sustituir ese identificador por el ARN del modelo aprovisionado que devuelve la operación `CreateProvisionedModelThroughput` en el parámetro `modelId` de la llamada de inferencia. Este caso ilustra un patrón recurrente en el examen: cuando un enunciado describe capacidad reservada infrautilizada junto con throttling simultáneo en producción, casi siempre la causa raíz es un error de enrutamiento en el parámetro `modelId`, no una insuficiencia real de capacidad ni un problema de resiliencia de la aplicación.

Fuentes

- <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-thru-use.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-thru-purchase.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-thru-prereq.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/custom-model-use-pt.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_CreateProvisionedModelThroughput.html
- <https://docs.aws.amazon.com/bedrock/latest/userguide/capacity-limits-cost-optimization.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html> ##

MÓDULO 2: RAG Y KNOWLEDGE BASES (BASES DE CONOCIMIENTO)

Retrieval-Augmented Generation (RAG) es, con 20 de las 97 preguntas del banco de práctica, el tema individual más examinado — empatado con Agentes. La razón es estructural: casi cualquier aplicación empresarial de IA Generativa necesita anclar sus respuestas en datos propios y actualizados sin reentrenar el modelo, y Amazon Bedrock Knowledge Bases ofrece tantas piezas configurables (almacén vectorial, estrategia de *chunking*, filtrado, *reranking*, descomposición de consultas, ingesta) que el examen puede construir un escenario distinto combinando cada vez una pieza diferente. Dominar este módulo significa entender **qué resuelve cada pieza y cuál es su límite exacto**, porque casi todos los distractores son técnicamente plausibles — simplemente no resuelven el requisito específico que pide el enunciado, o lo resuelven con más esfuerzo del necesario.



Flujo completo de un sistema RAG en Amazon Bedrock: ingestión (chunking + embeddings + almacén vectorial) y recuperación/generación (búsqueda híbrida + reranking + FM).

2.1 Qué resuelve Bedrock Knowledge Bases y qué no

Una Knowledge Base de Amazon Bedrock es el servicio totalmente gestionado que encapsula las cuatro etapas de un pipeline RAG — trocear documentos, generar embeddings, indexarlos en un almacén vectorial y orquestar la recuperación más la generación — sin que el equipo tenga que escribir ni mantener esa lógica. El patrón recurrente del examen es exactamente ese: cuando una opción propone “construir” alguna de esas etapas a mano (una función Lambda que llama a un modelo de embeddings, un endpoint de SageMaker para reranking, un pipeline con Step Functions), casi siempre es un distractor frente a la opción que activa la capacidad equivalente **ya integrada** en Knowledge Bases — salvo que el enunciado pida explícitamente algo que Knowledge Bases no soporta de forma nativa (una arquitectura de grafo complejísima, un modelo de embeddings de dominio muy específico entrenado a medida, etc.).

2.2 Elegir el almacén vectorial correcto

La pregunta “¿qué base de datos vectorial uso?” nunca se responde en abstracto: depende del volumen de datos, de si ya existe una base de datos relacional en la organización, de si se necesita precisión exacta o aproximada, y de si las relaciones entre entidades son tan importantes como la similitud semántica.

- **Amazon OpenSearch Serverless (Vector Engine)** es la opción por defecto y la más recurrente en el examen para colecciones grandes (millones de documentos): es *serverless* (AWS gestiona el aprovisionamiento y el escalado mediante OpenSearch Compute Units, OCU), soporta **filtrado nativo por metadatos** dentro de la propia consulta vectorial, y soporta **búsqueda híbrida** (ver 2.4). Es la respuesta esperada cuando el enunciado combina “millones de embeddings”, “múltiples idiomas” y “mínimo esfuerzo de gestión”.
- **Amazon Aurora PostgreSQL / RDS PostgreSQL con pgvector** (incluida la variante **Aurora Serverless v2**) es la elección natural cuando la empresa ya tiene datos relacionales en PostgreSQL y quiere unificar datos transaccionales con vectores semánticos en el mismo motor, o cuando el volumen es modesto (por debajo de ~1 millón de documentos) y se prioriza la simplicidad operativa de una base serverless sobre un motor de búsqueda dedicado. El matiz de examen: si la variante es la instancia **no-serverless** de Aurora/RDS, la opción pierde puntos frente a una alternativa serverless porque exige aprovisionar y dimensionar manualmente la instancia — justo lo que “mínimo esfuerzo operativo” penaliza.
- **Amazon Neptune Analytics (GraphRAG)** es la respuesta cuando el enunciado describe **relaciones multi-hop** entre entidades — dependencias indirectas entre vehículos de inversión, cadenas de suministro, redes de fraude — y pide “capturar relaciones indirectas” o “conectar información a través de varios pasos”.

lógicos”. GraphRAG, integrado de forma nativa en Bedrock Knowledge Bases, automatiza tres pasos: búsqueda vectorial inicial para localizar los nodos relevantes, recuperación de los nodos/chunks conectados a ellos, y expansión del contexto recorriendo el grafo — todo sin que el equipo programe ninguna lógica de recorrido. Si el escenario es una búsqueda semántica simple sin relaciones entre entidades, Neptune Analytics es sobreingeniería: la respuesta correcta suele ser un almacén vectorial convencional.

- **Amazon MemoryDB (Valkey)** aparece cuando el requisito es **maximizar la precisión** sobre un dataset **pequeño** (bajo recuento de índice): el algoritmo **Flat** (búsqueda por fuerza bruta, k-NN exacto) es la respuesta correcta en ese caso concreto, porque el coste de un recorrido lineal es aceptable a pequeña escala y se obtiene precisión exacta con latencia mínima gracias a que MemoryDB es en memoria. El algoritmo **HNSW** (aproximado) es preferible a mayor escala, pero sacrifica algo de precisión — si el enunciado pide explícitamente “maximizar precisión” sobre un índice pequeño, HNSW es el distractor, no la respuesta.
- **Amazon DocumentDB y Aurora PostgreSQL con IVFFlat** son también motores de búsqueda **aproximada** (ANN): IVFFlat particiona el espacio vectorial en listas y compara solo un subconjunto, lo que introduce pérdida de recall frente a un método exacto como Flat.
- **Amazon S3 Vectors** es una opción relativamente nueva con dos tipos de metadatos: **filterable** (el comportamiento por defecto, permite filtrar en la consulta de similitud) y **non-filterable** (se devuelven junto al resultado pero **no** se pueden usar como filtro, y esa marca es irreversible). Si un escenario exige filtrar por fecha, agencia o tipo de documento y la opción configura esos campos como *non-filterable*, esa opción es incorrecta aunque S3 Vectors en sí sea una integración válida con Knowledge Bases.
- **Terceros integrados de forma nativa:** Pinecone, Qdrant y Redis Enterprise también son almacenes vectoriales soportados cuando la organización ya opera con ellos.

2.3 Estrategias de chunking y un matiz que el examen explota

El tamaño y la estrategia de *chunking* determinan directamente la calidad semántica de la recuperación, y el examen prueba tanto el **cuándo usar cada estrategia** como una restricción operativa poco intuitiva.

Estrategia	Cómo funciona	Cuándo es la respuesta correcta
Fixed-size (por defecto)	Bloques de N tokens (300 por defecto, ~20% de overlap).	Documentos homogéneos y sencillos. Es el distractor típico cuando el problema real es que corta argumentos o cláusulas a la mitad.
Hierarchical (Parent-Child)	Chunks hijo pequeños para precisión de búsqueda; se recupera el chunk padre completo como contexto.	Cuando se necesita contexto amplio para síntesis, pero el escenario no exige agrupar por <i>significado</i> — solo por tamaño en dos niveles.
Semantic	Divide por similitud coseno entre oraciones adyacentes; parámetros <code>breakpointPercentileThreshold</code> y <code>bufferSize</code> .	La respuesta preferida cuando el problema descrito es “se pierde contexto clave” o “se citan datos desactualizados/incompletos”

Estrategia	Cómo funciona	Cuándo es la respuesta correcta
		porque el chunking de tamaño fijo corta argumentos legales, cláusulas o secciones a la mitad. Prioriza el significado sobre la posición del texto.
Custom (AWS Lambda)	Lógica propietaria invocada durante la sincronización.	Formatos altamente estructurados (tablas complejas, XML/JSON propietario) que las estrategias estándar destruyen.

⚠ Regla de Examen

La estrategia de chunking de una Knowledge Base **no se puede modificar después de conectar la fuente de datos**. Si un escenario pide corregir un problema de fragmentación semántica, la solución correcta casi siempre implica **reconfigurar la ingesta desde cero y regenerar los embeddings** con la nueva estrategia — no un ajuste en caliente. Aumentar la dimensionalidad del embedding o migrar el almacén vectorial sin tocar el chunking nunca corrige un problema de fragmentación: la causa raíz sigue intacta.

2.4 Metadata filtering, búsqueda híbrida y reranking: tres capas de precisión distintas

Estas tres técnicas se confunden fácilmente porque todas “mejoran la relevancia”, pero atacan problemas diferentes y el examen exige identificar cuál corresponde a cada síntoma:

- **Metadata filtering** reduce el *universo* de documentos sobre los que se calcula similitud, **antes** de la búsqueda vectorial. Con una fuente de datos en S3, se implementa adjuntando a cada objeto un archivo complementario `nombre-archivo.metadata.json` con atributos (tipo de contenido, fecha, autor); una vez indexados, se filtra por esos atributos en `Retrieve / RetrieveAndGenerate`. Es la respuesta correcta cuando el síntoma es “la búsqueda vectorial evalúa demasiados documentos irrelevantes de tipos o periodos distintos” y el requisito es un cambio mínimo sobre la arquitectura existente — no exige migrar de almacén vectorial ni reentrenar embeddings.
- **Hybrid search** combina, dentro de una misma consulta, la puntuación léxica (BM25, exacta) con la puntuación semántica (vectorial/k-NN), normalizando y combinando ambas (por ejemplo con `min_max / l2` y media aritmética, geométrica o armónica). Es la respuesta correcta cuando el síntoma es que la búsqueda **pierde coincidencias exactas** de términos, acrónimos, códigos de error o citas legales — algo que la búsqueda puramente vectorial “diluye” semánticamente. Se activa como un *search pipeline* sobre el mismo dominio de OpenSearch ya existente, sin infraestructura nueva.
- **Reranking** actúa **después** de la recuperación inicial: sobre los primeros k resultados (por ejemplo, 25), un modelo de reranking dedicado (Amazon Rerank o Cohere Rerank) reordena por relevancia contextual estricta y se pasan solo los 3-5 mejores al FM. La forma de menor esfuerzo operativo es habilitarlo **directamente en la configuración de la Knowledge Base** (`rerankingConfiguration` dentro de

`vectorSearchConfiguration`, disponible en `Retrieve / RetrieveAndGenerate / RetrieveAndGenerateStream`) — llamar a la **API Rerank** por separado como un paso adicional también funciona, pero obliga a orquestar manualmente varios pasos (`Retrieve` → `Rerank` → `InvokeModel`) en lugar de una sola configuración integrada.

Caso de Estudio — Tres síntomas, tres soluciones distintas

Tres escenarios aparentemente parecidos requieren tres respuestas distintas: (1) si el síntoma es que la búsqueda evalúa demasiados documentos de tipos y periodos irrelevantes y se pide un cambio mínimo, la respuesta es **habilitar filtrado por metadatos** indexando los atributos S3; (2) si el síntoma es que se pierden coincidencias exactas de acrónimos o términos técnicos, la respuesta es **búsqueda híbrida**; (3) si el síntoma es que la recuperación inicial devuelve documentos semánticamente parecidos pero contextualmente irrelevantes y se pide el mínimo esfuerzo operativo, la respuesta es **activar el reranking nativo dentro de la configuración de la Knowledge Base**, no desplegar un modelo de ranking en SageMaker ni orquestar la API Rerank por separado.

2.5 Query Decomposition: cuando la pregunta del usuario es el problema

Además de mejorar cómo se buscan los documentos, Bedrock Knowledge Bases puede transformar la propia consulta antes de recuperar: la transformación **QUERY_DECOMPOSITION** (configurable vía `QueryTransformationConfiguration`) divide automáticamente una pregunta compleja en subconsultas más simples, cada una enfocada en un concepto o término específico, antes de ejecutar la recuperación. Esto reduce la **dilución semántica** — el efecto por el que una pregunta que mezcla varios conceptos técnicos recupera fragmentos genéricos que no cubren bien ninguno de ellos — sin necesidad de orquestar manualmente dos modelos (uno que descomponga, otro que expanda) ni de construir esa lógica con agentes o funciones Lambda personalizadas. Combinada con **Amazon Bedrock Flows** (un nodo de FM más un nodo de Knowledge Base conectados visualmente), resuelve con mínimo esfuerzo operativo el patrón “consulta compleja de dominio muy especializado, alto volumen, baja latencia”.

2.6 Citas, explicabilidad y por qué RetrieveAndGenerate gana casi siempre

Cuando el requisito es que la aplicación **explique su razonamiento y cite las fuentes**, la respuesta con menor esfuerzo operativo es prácticamente siempre habilitar **RetrieveAndGenerate** (o su variante en streaming, `RetrieveAndGenerateStream`): la operación devuelve la respuesta generada junto con objetos `Citation` que enlazan cada afirmación con los fragmentos de origen exactos, sin que la aplicación tenga que comparar manualmente el texto generado contra los documentos recuperados. Construir esa comparación a mano (opción recurrente entre los distractores: usar `Retrieve` + `InvokeModel` por separado, o post-procesar con Amazon Comprehend Medical, o con Amazon Kendra más lógica propia) siempre implica más desarrollo para lograr un resultado que la API nativa ya ofrece integrado.

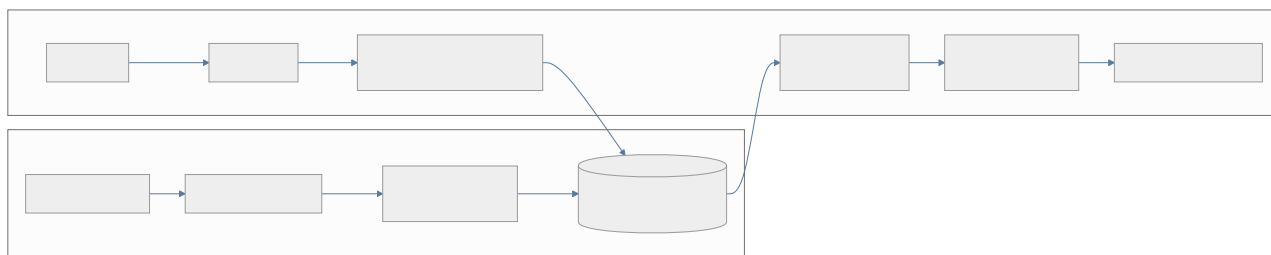
2.7 Reducir alucinaciones: RAG no es magia por sí solo

Un escenario recurrente describe un modelo que “inventa” productos, datos o citas inexistentes. La combinación de examen correcta combina dos palancas complementarias: (1) **RAG con chunking semántico y embeddings bien ajustados**, que ancla la generación en fragmentos de origen realmente relevantes en lugar de depender de lo aprendido en el entrenamiento; y (2) **instrucciones explícitas de razonamiento paso a paso con verificación de hechos** (chain-of-thought zero-shot) antes de emitir la respuesta final. Lo que **no** reduce alucinaciones, y aparece sistemáticamente como distractor: subir la `temperature` (aumenta la aleatoriedad, el efecto contrario); los *content filters* de Guardrails (detectan categorías de daño predefinidas — odio, violencia — no “patrones de alucinación”, que no existen como tal); y seguir resumiendo el documento completo en una sola pasada sin recuperación selectiva. El mecanismo real de Bedrock para detectar alucinaciones frente a una fuente de referencia es el **Contextual Grounding Check** de Guardrails (ver Módulo 4): calcula una puntuación de *grounding* (fidelidad a la fuente) y de *relevance* (pertinencia a la pregunta), no un filtro de “patrones”.

2.8 Patrones de arquitectura multi-cuenta, multi-región y de ingesta

Varios escenarios de examen combinan RAG con requisitos de aislamiento, residencia de datos o actualización en tiempo casi real — la solución rara vez es una única Knowledge Base gigante:

- **Aislamiento por inquilino/unidad de negocio:** cuando cada hotel, cliente o departamento necesita **controles de acceso separados y rendimiento aislado** (para que un pico de uso de uno no degrade a los demás), la respuesta es **una Knowledge Base por cuenta** dentro de una estructura multicuenta — usando **políticas de recursos (resource-based policies)** de Knowledge Bases gestionadas para permitir consultas cross-account (`bedrock:Retrieve` , `bedrock:GetDocumentContent`) cuando haga falta compartir. Una única KB combinada, aunque tenga filtros de acceso, sigue compartiendo las mismas cuotas de servicio para todos los inquilinos.
- **Residencia geográfica con baja latencia global:** la combinación correcta es una **Knowledge Base local por región** (los documentos y el almacén vectorial nunca salen de su región de origen) más un **inference profile de Cross-Region Inference geográfico** para el modelo generador — el perfil geográfico solo cubre el enrutamiento del modelo, no dónde vive el índice vectorial ni los documentos fuente; ambas piezas son necesarias.
- **Actualización casi en tiempo real:** además de la sincronización programada convencional (`StartIngestionJob`), Knowledge Bases admite **ingesta directa** de documentos individuales mediante un *data source* de tipo *Custom* y operaciones como `IngestKnowledgeBaseDocuments` , sin esperar a un trabajo de sincronización completo. La combinación típica de examen es: ingesta directa para el dato que cambia con frecuencia (p. ej. disponibilidad de habitaciones) y sincronización programada para el resto (políticas, descripciones).
- **Caché semántica** (reducir llamadas redundantes al modelo cuando muchas preguntas de usuario son reformulaciones de la misma idea): se genera un embedding de cada consulta entrante y se busca por similitud (k-NN aproximado, por ejemplo en OpenSearch) contra un índice de preguntas-respuestas ya resueltas, sirviendo la respuesta cacheada por encima de un umbral de similitud coseno configurable. Cachés basadas en coincidencia exacta de texto (DynamoDB con `LIKE` , *stemming*, índices por texto normalizado) no detectan reformulaciones semánticamente equivalentes con vocabulario distinto.



Fuentes

- <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-setup.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-it-works.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-customize-ingestion.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-retrieve-generate.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-ds-s3.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-cross-account.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-direct-ingestion.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-build-graphs.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/rerank-use.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/rerank.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>
- <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.VectorDB.html>
- <https://docs.aws.amazon.com/memorydb/latest/devguide/vector-search-overview.html>
- <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/vector-search.html>
- <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/semantic-search.html>
- <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/serverless-configure-neural-search.html>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors-metadata-filtering.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/flows-nodes.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_QueryTransformationConfiguration.html ## MÓDULO 3: AGENTES AUTÓNOMOS Y ORQUESTACIÓN (AGENTS & FLOWS)

Este módulo es, junto con RAG, el que más peso tiene en el examen real (20 de 97 preguntas analizadas). Y el patrón es claro: el examen no se limita a preguntar “qué es un Bedrock Agent”. Pregunta, escenario tras escenario, algo mucho más útil en la práctica: **cuándo un Agente es la herramienta correcta, y cuándo no lo es** — cuándo conviene sustituirlo por una orquestación determinista con AWS Step Functions, por configura-

ción dinámica con AWS AppConfig, o por la plataforma gestionada Amazon Bedrock AgentCore. Este capítulo cubre primero la anatomía clásica de un Agente y después la capa de orquestación de producción que rodea a los agentes en las arquitecturas reales que examina AWS.

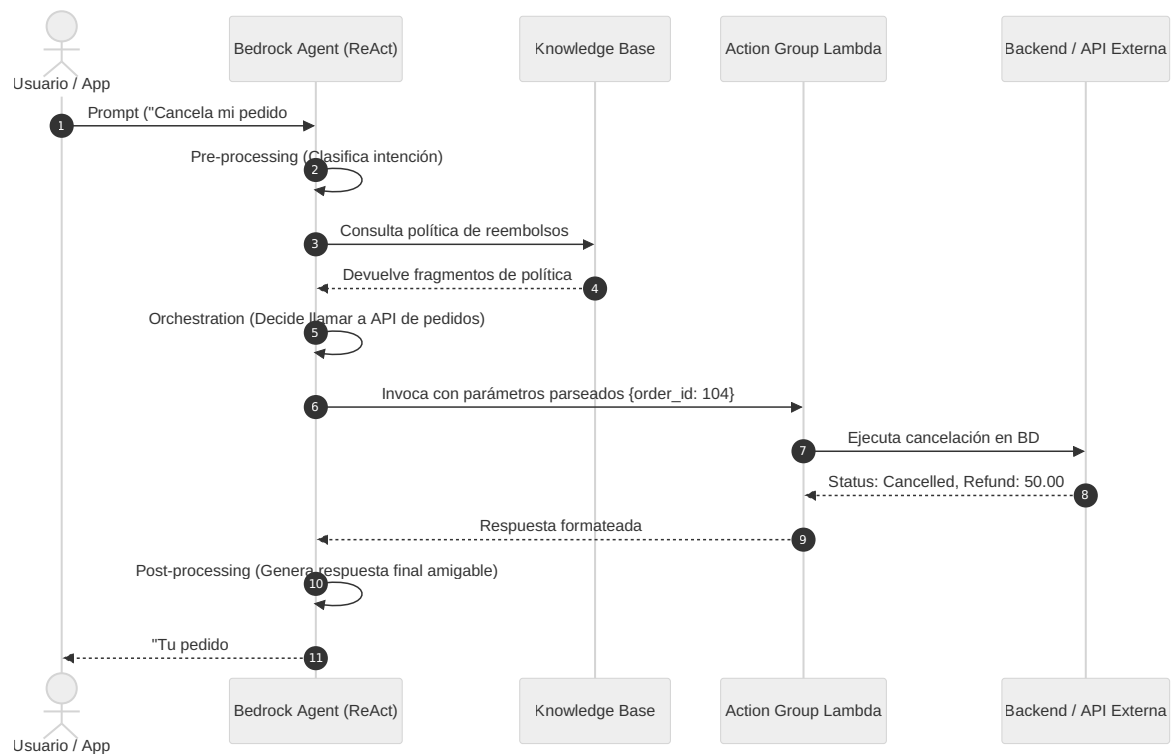
3.1 El patrón ReAct y el ciclo de vida de un Bedrock Agent

Un **Amazon Bedrock Agent** extiende un modelo fundacional (FM) con la capacidad de razonar en varios pasos y actuar sobre el mundo exterior, siguiendo el patrón **ReAct (Reasoning + Acting)**: el modelo alterna entre pensar (“¿qué necesito hacer para responder a esto?”) y actuar (invocar una herramienta, consultar una base de conocimiento), incorporando el resultado de cada acción a su siguiente razonamiento, hasta que dispone de suficiente información para responder.

AWS documenta el ciclo de ejecución de un agente en tres fases, y cada una emite sus propios eventos de traza:

1. **Pre-processing:** el agente valida y contextualiza la entrada del usuario, clasificando la intención antes de razonar sobre ella.
2. **Orchestration:** el núcleo ReAct. El FM decide, en cada iteración, si necesita invocar una acción de un *action group*, consultar una *knowledge base* asociada, o si ya tiene suficiente información para responder. Cada decisión genera un *rationale* (el razonamiento explícito del modelo).
3. **Post-processing:** el agente formatea la respuesta final para el usuario, opcionalmente aplicando una plantilla o resumen.

Cada una de estas fases produce un tipo de evento de traza (`PreProcessingTrace` , `OrchestrationTrace` , `PostProcessingTrace`), y una cuarta traza, `FailureTrace` , documenta errores de invocación (fallos de Lambda, esquemas OpenAPI inválidos). Estas trazas son la base de la trazabilidad y la auditoría del comportamiento del agente — ver Módulo 5.



Ciclo de ejecución de un Amazon Bedrock Agent con ReAct

3.2 Anatomía de un Agent: FM, Instruction, Action Groups, Knowledge Bases

Un agente se compone de cuatro elementos:

- **Foundation Model:** el motor de razonamiento (por ejemplo, Claude 3.5 Sonnet). Determina la calidad del razonamiento multi-paso y la fiabilidad de la selección de herramientas.
- **Instruction:** el *system prompt* del agente — define su rol, tono, restricciones y objetivos. Es el lugar donde se acotan comportamientos (“nunca reveles el esquema interno de la base de datos”, “responde siempre en el idioma del usuario”).
- **Action Groups:** el conjunto de acciones que el agente puede ejecutar en el mundo real. Se describen mediante un **esquema OpenAPI** (JSON o YAML, alojado en S3) que enumera los *endpoints*, verbos HTTP, parámetros obligatorios/opcionales y tipos de datos de cada acción. El agente usa las descripciones en lenguaje natural de cada operación del esquema para decidir, en tiempo de razonamiento, qué acción invocar y con qué parámetros.
- **Knowledge Bases asociadas:** cuando la pregunta del usuario es informativa en lugar de transaccional, el agente puede consultar automáticamente una o más *knowledge bases* mediante RAG, sin que el desarrollador tenga que programar esa decisión explícitamente — el propio razonamiento del agente decide si la pregunta requiere consulta documental.

3.3 Ejecución de acciones: AWS Lambda vs Return of Control (RoC)

Un *action group* necesita un mecanismo que traduzca la decisión del agente en una ejecución real. Existen dos:

- **AWS Lambda Function:** el mecanismo por defecto. Bedrock invoca automáticamente la función Lambda asociada al *action group*, le pasa los parámetros que el modelo ha extraído del lenguaje natural del usuario, y recibe de vuelta el resultado para continuar el razonamiento.
- **Return of Control (RoC):** Bedrock *no* ejecuta nada por sí mismo. En su lugar, devuelve a la aplicación cliente el nombre de la función y los parámetros estructurados que el modelo ha decidido invocar, y espera a que la aplicación ejecute la acción y le devuelva el resultado en la siguiente llamada a `InvokeAgent`.

La razón de examen para elegir RoC es siempre la misma: **el recurso que hay que invocar no es accesible desde la cuenta de AWS del agente** — está on-premises, detrás de un firewall corporativo, o requiere credenciales que la aplicación cliente posee pero que no se deben exponer nunca a la infraestructura de Bedrock (por ejemplo, tokens de sesión de un sistema bancario interno). RoC traslada la responsabilidad de ejecución — y de gestión de esas credenciales sensibles — al entorno del cliente, mientras que Lambda es la opción natural cuando el recurso ya es accesible desde la cuenta de AWS.

3.4 Memoria: de la sesión con estado a Amazon Bedrock AgentCore

Un agente básico distingue dos niveles de memoria:

- **Sesión a corto plazo:** el contexto de la conversación en curso, identificado por un `sessionId`, con un tiempo de vida configurable (por defecto, una hora).
- **Memoria a largo plazo:** un resumen automático de sesiones anteriores del mismo usuario, que permite personalizar interacciones futuras conservando preferencias históricas sin tener que releer el historial completo.

El examen va, sin embargo, un paso más allá de esta memoria básica de los Bedrock Agents “clásicos”: pregunta por **Amazon Bedrock AgentCore**, la plataforma agéntica gestionada de AWS pensada para llevar agentes a producción a gran escala. AgentCore no es un único servicio, sino un conjunto de capacidades independientes y componibles:

- **AgentCore Memory:** gestiona memoria a corto y largo plazo de forma nativa, compatible entre múltiples agentes y persistente entre sesiones — sin que el desarrollador tenga que diseñar su propio esquema de DynamoDB o Aurora para el estado conversacional.
- **AgentCore Identity:** gestión de identidad, autenticación y autorización, con soporte para proveedores externos (Amazon Cognito, Okta, Microsoft Entra ID).
- **AgentCore Runtime:** aísla cada sesión en una microVM dedicada, y expone tanto invocación síncrona (API `InvokeAgentRuntime`) como procesamiento asíncrono/orientado a eventos.
- **AgentCore Gateway:** convierte APIs existentes (REST APIs vía Amazon API Gateway, funciones Lambda) en herramientas compatibles con el **Model Context Protocol (MCP)** con mínima configuración, sin escribir código de orquestación personalizado — centraliza el descubrimiento de herramientas y la autenticación de entrada/salida.
- **AgentCore Observability:** trazabilidad integrada del comportamiento del agente.

Caso de Estudio — Memoria, identidad y herramientas sin construir infraestructura

Un equipo construye un agente de soporte con AWS Lambda y APIs REST que debe preservar memoria entre interacciones, compartir estado entre agentes, soportar invocación tanto síncrona como orientada a eventos, y aplicar control de acceso por sesión — con el mínimo esfuerzo de desarrollo. La tentación es diseñarlo todo a mano: Step Functions más colas SQS más una tabla DynamoDB para el estado. Es una arquitectura válida, pero exige construir y mantener manualmente exactamente las capacidades — memoria, identidad, aislamiento de sesión, observabilidad — que **Amazon Bedrock AgentCore** ya ofrece como servicio gestionado. La combinación ganadora en el examen es AgentCore para memoria/identidad/runtime, más **AgentCore Gateway** para exponer las Lambdas y las REST APIs existentes como herramientas MCP sin escribir orquestación adicional. Cualquier alternativa que reconstruya estas piezas a mano (ECS + Aurora, RAG casero con Kendra + S3 para el estado) es funcionalmente posible pero pierde frente a la opción gestionada en “menor esfuerzo operativo”.

3.5 Multi-Agent Collaboration: arquitectura jerárquica y aislamiento por dominio

Cuando un único agente debe cubrir varios dominios de negocio (facturación, soporte técnico, reservas, o en un caso clínico: consultas médicas, verificación de seguros, citas y reclamaciones), la arquitectura documentada por AWS es siempre **jerárquica**: un **Supervisor Agent** recibe la consulta del usuario, clasifica su intención en lenguaje natural, y la enruta al **Subagente colaborador** especializado en ese dominio. Cada colaborador — incluido el propio supervisor— puede tener sus propias herramientas, *action groups*, *knowledge bases* y *guardrails*.

El matiz que el examen exige dominar es la dirección del enrutamiento y el aislamiento de datos: es **el supervisor** quien clasifica y enruta (nunca al revés, y nunca mediante supervisores paralelos que fusionan respuestas por lógica externa), y cada colaborador debe tener **su propia knowledge base aislada**, con el acceso controlado mediante políticas de IAM específicas por agente/base de conocimiento. Esto permite añadir un nuevo dominio de negocio (un nuevo departamento, una nueva especialidad) sin tocar ni volver a certificar la lógica de los demás colaboradores — la razón por la que esta arquitectura escala mejor que un único agente monolítico con múltiples *action groups* y reglas de enrutamiento incrustadas en sus instrucciones, que se vuelve cada vez más difícil de mantener a medida que crecen los dominios.

3.6 Amazon Bedrock Flows: orquestación visual y determinista

Amazon Bedrock Flows es el complemento determinista de los Agents: un entorno visual para definir un grafo dirigido de nodos donde el flujo de ejecución —a diferencia del razonamiento autónomo de un agente— es fijo y predecible. Los tipos de nodo relevantes para el examen son:

- **Prompt node**: invoca un modelo de Bedrock con una plantilla de prompt parametrizada — el nodo correcto para una simple tarea de inferencia (por ejemplo, “revisa este texto y sugiere mejoras”).
- **Knowledge Base node**: ejecuta una consulta RAG contra una base de conocimiento.
- **Condition / Branching node**: lógica condicional basada en reglas explícitas.
- **Lambda node**: transformación de datos o ejecución de código arbitrario.

- **S3 storage / retrieval node:** lee o escribe contenido directamente en un bucket de S3 dentro del propio flujo, sin necesidad de una Lambda intermedia.
- **Agent node:** invoca un Agente de Bedrock *completo*, con toda su orquestación, memoria y herramientas propias.

El matiz de examen más fino aquí es no confundir el **nodo Prompt** con el **nodo Agent**: si el requisito es simplemente “enviar un fragmento de texto a un modelo para que lo evalúe o transforme” dentro de un flujo por lo demás fijo (leer de S3 → parsear → invocar el modelo → escribir en S3), el nodo correcto es **Prompt**, no **Agent** — usar un nodo de agente para una inferencia puntual introduce razonamiento autónomo, latencia y coste innecesarios donde solo se necesita una llamada directa al modelo.

3.7 Cuándo NO usar un Agente: orquestación determinista con AWS Step Functions

Esta es la lección más repetida del banco de preguntas real: **no todo flujo generativo necesita un Agente**. Cuando la secuencia de pasos es fija y conocida de antemano (leer S3 → transcribir audio → construir un prompt → invocar el modelo → escribir el resultado), añadir el razonamiento no determinista de un agente solo introduce coste, latencia y comportamiento impredecible. La alternativa recomendada por AWS en estos casos es **AWS Step Functions**, con varias piezas que el examen evalúa en detalle:

- **Standard vs Express workflows:** los flujos **Standard** están pensados para procesos duraderos y auditables (hasta un año de ejecución), con semántica *exactly-once* y un historial de ejecución completo recuperable vía API — la opción correcta siempre que se necesite el patrón *Wait for a Callback with Task Token* (pausar el flujo indefinidamente en espera de una aprobación humana o de un reentrenamiento largo) o una auditoría completa. Los flujos **Express** están pensados para alto volumen y corta duración (máximo cinco minutos), con semántica *at-least-once* y sin historial completo — adecuados para tareas rápidas y deterministas como leer-transformar-invocar-escribir, pero *nunca* para esperar una respuesta humana ni para orquestar entrenamientos largos.
- **Estado `Parallel`:** ejecuta varias ramas del flujo de trabajo **simultáneamente**. Es la respuesta correcta cuando varias llamadas independientes al modelo (por ejemplo, análisis financiero, de sentimiento y de cumplimiento sobre el mismo informe) se pueden lanzar en paralelo para que el tiempo total sea el de la rama más lenta, no la suma de todas — la única forma de bajar de, por ejemplo, 45 a 10 segundos por lote.
- **Patrón `waitForTaskToken` (“Wait for a Callback with the Task Token”):** genera un token único, pausa la ejecución sin consumir cómputo, y la reanuda cuando un proceso externo (una función Lambda tras una revisión humana) llama a `SendTaskSuccess` o `SendTaskFailure` con ese token. Es el mecanismo nativo para flujos de **aprobación humana** (“human-in-the-loop”) y para aclaraciones conversacionales ambiguas, sin construir un sistema de *polling* a mano.
- **Manejo de errores declarativo (`Retry` / `Catch`):** cada estado admite políticas de reintento (con *backoff* exponencial configurable) y ramas de captura hacia un estado de *fallback* cuando los reintentos se agotan — la base de la “degradación agraciada” (*graceful degradation*) sin escribir lógica de reintento a mano.
- **Integración nativa con Amazon Bedrock:** el recurso `arn:aws:states:::bedrock:invokeModel` invoca un modelo directamente desde un estado `Task`, sin una Lambda intermedia. Admite los campos opcionales `Input.S3Uri` y `Output.S3Uri`: si el *payload* (`Body`) supera el límite de 256 KiB de Step Functions, se

recomienda leer la entrada y escribir la salida directamente en Amazon S3, pasando entre estados solo una referencia (URI) mediante `ResultPath` / `ResultSelector` en vez de la carga completa.

- **Disparo desde Amazon S3:** Step Functions no puede iniciarse directamente desde una notificación de evento de S3 (que solo soporta SNS, SQS, Lambda o EventBridge como destino nativo); el patrón documentado y obligatorio es configurar el bucket para enviar eventos a **Amazon EventBridge** y que una regla de EventBridge dispare la ejecución de la máquina de estados.

Caso de Estudio — Paralelismo real frente a “secuencial disfrazado”

Un sistema de análisis de informes financieros debe bajar de 45 a 10 segundos por lote, donde cada informe requiere varias llamadas independientes al modelo (datos financieros, sentimiento, cumplimiento). Varias opciones prometen “escalar” el procesamiento — concurrencia aprovisionada en Lambda, colas SQS con Lambdas de concurrencia reservada, un clúster ECS con auto-scaling — pero todas ellas, leídas con atención, siguen describiendo un procesamiento **secuencial** de los distintos tipos de análisis dentro de cada lote o informe: la suma de latencias no cambia. La única solución que rompe la cadena es un estado **Parallel** de Step Functions, que invoca una rama independiente por cada tipo de análisis de verdad en paralelo, reduciendo el tiempo total al de la rama más lenta. La lección de examen: cuando una opción usa la palabra “paralelo” o “simultáneo” pero la letra pequeña dice “sequentially”, es un distractor.

3.8 Configuración dinámica sin redespiegue: AWS AppConfig

Un patrón que aparece repetidamente en el banco de preguntas: una aplicación necesita cambiar en tiempo real las reglas de negocio que deciden qué modelo invocar (por umbrales de coste, nivel de cliente, región regulatoria, tráfico de pruebas A/B) **sin desplegar código nuevo**, y con capacidad de validar y revertir esos cambios de forma segura. La respuesta casi siempre es **AWS AppConfig** (una capacidad de AWS Systems Manager), diseñado exactamente para separar la configuración dinámica del código:

- **Feature flags nativos** para pruebas A/B y activación/desactivación de funcionalidades sin redespiegue.
- **Validadores** (JSON Schema o Lambda personalizado) que comprueban la validez de la configuración *antes* de desplegarla — evita que un valor de `temperature` o un límite de tokens mal formado llegue a producción.
- **Estrategias de despliegue** (por ejemplo, `Linear20PercentEvery6Minutes`) para un *rollout* gradual y controlado del cambio de configuración.
- **Rollback automático** integrado con Amazon CloudWatch: si una alarma entra en estado `ALARM` durante el despliegue, AppConfig revierte automáticamente a la versión anterior.
- **AWS AppConfig Agent (extensión de Lambda):** añadido como capa (*layer*) a la función, mantiene una caché local de la configuración accesible vía un *endpoint* HTTP local (`localhost:2772`), reduce las llamadas al servicio y la latencia, y comprueba periódicamente si hay actualizaciones.

El matiz de examen frente a los distractores habituales: **AWS Systems Manager Parameter Store** y **Amazon DynamoDB** permiten almacenar y leer configuración en tiempo de ejecución sin redespiegue, pero *no* ofrecen de forma nativa validadores de esquema, estrategias de *rollout* gradual ni *rollback* automático basado en alarmas — toda esa lógica de seguridad habría que construirla a mano. **Amazon ElastiCache** añade la sobrecarga de gestionar un clúster de caché para un problema que AppConfig resuelve de forma gestionada. Y almacenar

reglas en variables de entorno de Lambda o en *stage variables* de API Gateway obliga, en la práctica, a una actualización de configuración o un despliegue de etapa cada vez que cambian las reglas — justo lo que el requisito de “sin redesplicue” prohíbe.

3.9 Model Context Protocol (MCP) en producción sobre AWS

El **Model Context Protocol (MCP)** estandariza cómo un agente descubre e invoca herramientas externas. El examen distingue con precisión dos transportes y dos patrones de despliegue:

- **STDIO:** transporte para procesos **locales** que se ejecutan en la misma máquina (entrada/salida estándar) — no está pensado ni sirve para invocar de forma remota un servicio en la nube como una función Lambda.
- **Streamable HTTP:** transporte para servidores MCP **remotos**, basado en JSON-RPC 2.0 sobre peticiones HTTP POST con soporte de streaming — el transporte obligatorio cuando el servidor MCP se expone como un servicio accesible por red.

Para desplegar un servidor MCP sobre AWS Lambda accesible tanto a aplicaciones internas como a socios externos con controles estrictos de autenticación, existen dos patrones válidos, con un criterio de desempate claro:

- **Lambda function URLs** con `InvokeMode = RESPONSE_STREAM` (habilita el streaming que exige MCP) y `AuthType = AWS_IAM` (autenticación mediante firma SigV4, con permisos `lambda:InvokeFunctionUrl` otorgados en políticas de identidad o, para terceros externos, en la política de recursos de la función). Es la opción de **menor sobrecarga operativa** cuando no se necesitan las funcionalidades adicionales de API Gateway (planes de uso, dominios personalizados, autorizadores complejos).
- **Amazon API Gateway (HTTP API)** con Streamable HTTP y **Amazon Cognito** para emitir y validar tokens OAuth 2.1 — arquitectura igualmente válida y más rica en funcionalidades, pero con más componentes que aprovisionar y mantener.

Tanto las claves de API de API Gateway como el transporte STDIO quedan descartados en el examen para este tipo de escenario: las API keys son un mecanismo de identificación/limitación de tasa, no de autenticación fuerte de usuarios finales, y STDIO no es un transporte de red. **Amazon Bedrock AgentCore Gateway** (sección 3.4) ofrece, además, una vía gestionada para exponer APIs y Lambdas existentes como herramientas MCP sin construir manualmente ninguna de estas capas de transporte y autenticación.

Caso de Estudio — El transporte correcto para el problema correcto

Un GenAI Developer debe exponer un servidor MCP (hoy ejecutándose localmente en los portátiles de los desarrolladores) como una función Lambda en producción, accesible para aplicaciones internas y socios externos autorizados, con controles estrictos de autenticación. La trampa más común es reutilizar STDIO “porque ya funcionaba en local”: STDIO solo tiene sentido para procesos en la misma máquina, nunca para invocar una Lambda remota. La segunda trampa es asumir que cualquier HTTP vale: usar API Gateway con claves de API sustituye MCP por peticiones HTTP genéricas y ofrece una autenticación débil. La solución de mínimo esfuerzo operativo es Lambda function URLs con transporte Streamable HTTP e IAM/SigV4 — sin aprovisionar ni mantener Amazon API Gateway.

3.10 Elegir la herramienta correcta: Agent, Flow o Step Functions

La pregunta que subyace a la mayoría de los escenarios de este módulo es una sola: **¿esta tarea necesita razonamiento autónomo, o es una secuencia de pasos conocida de antemano?**

- Si el sistema necesita **decidir dinámicamente** qué acción tomar, interpretar lenguaje natural ambiguo, o elegir entre múltiples herramientas según el contexto de la conversación → **Bedrock Agent** (o **Multi-Agent Collaboration** si hay varios dominios especializados).
- Si el flujo es una secuencia **fija y conocida** de transformación de datos (leer → transformar → invocar el modelo → escribir), con necesidad de paralelismo real, reintentos declarativos, pausas de aprobación humana o integración nativa con decenas de servicios de AWS → **AWS Step Functions**.
- Si se necesita un **editor visual** para una tubería generativa moderadamente compleja pero determinista, con nodos de prompt, RAG, condición y Lambda, sin construir una máquina de estados a mano → **Amazon Bedrock Flows**.
- Si la variable que cambia con frecuencia es la **configuración de negocio** (qué modelo usar, qué umbral aplicar, qué variante de prueba A/B), no la lógica de orquestación en sí → **AWS AppConfig**, combinado con cualquiera de las anteriores.

Caso de Estudio — Cuando el agente es la sobreingeniería

Una aplicación necesita, ante un evento de revisión de comunicaciones al cliente, leer una plantilla de S3, extraer una porción concreta, enviarla a un modelo de Bedrock para revisión, y guardar la respuesta de vuelta en el mismo bucket. Es tentador resolverlo con un Agente de Bedrock con un *action group* (o incluso tres funciones Lambda dentro de ese *action group*) que “razone” sobre cómo hacerlo. Pero el flujo es, en esencia, cuatro pasos fijos: leer, parsear, invocar, escribir. Introducir un agente añade una capa de razonamiento no determinista, más lenta y más cara, a un proceso que no necesita ninguna decisión. La solución de examen es un flujo Express de **AWS Step Functions** con integraciones de servicio nativas: `s3:getObject`, un estado `Pass` con funciones intrínsecas para parsear, `bedrock:InvokeModel`, y `s3:putObject` — sin una sola línea de código Lambda.

Fuentes

- <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/trace-events.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-sync-ingest.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/create-multi-agent-collaboration.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/flows-nodes.html>
- <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html>
- <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway.html>
- <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-sessions.html>
- <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-mcp.html>
- <https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html>

- <https://docs.aws.amazon.com/step-functions/latest/dg/state-parallel.html>
 - <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-error-handling.html>
 - <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-human-approval.html>
 - <https://docs.aws.amazon.com/step-functions/latest/dg/input-output-resultpath.html>
 - <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-cloudwatch-events-s3.html>
 - <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/step-functions-workflows.html>
 - <https://docs.aws.amazon.com/appconfig/latest/userguide/what-is-appconfig.html>
 - <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-integration-lambda-extensions.html>
 - <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-creating-deployment-strategy.html>
 - <https://docs.aws.amazon.com/appconfig/latest/userguide/appconfig-creating-configuration-and-profile-validators.html>
 - <https://docs.aws.amazon.com/appconfig/latest/userguide/monitoring-deployments.html>
 - <https://docs.aws.amazon.com/lambda/latest/dg/urls-auth.html>
 - <https://docs.aws.amazon.com/lambda/latest/dg/response-streaming-tutorial.html>
 - <https://docs.aws.amazon.com/lambda/latest/dg/apig-http-invoke-decision.html>
 - <https://docs.aws.amazon.com/prescriptive-guidance/latest/mcp-strategies/mcp-hosting-strategy.html>
 - <https://docs.aws.amazon.com/prescriptive-guidance/latest/mcp-deployment-patterns-on-aws/deployment-pattern-2-aws-lambda-amazon-api-gateway.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-generate-query.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html> ##
- MÓDULO 4: GUARDRAILS, SEGURIDAD Y FILTRADO DE CONTENIDO

4.1 Guardrails como capa de seguridad independiente del modelo

Amazon Bedrock Guardrails es, conceptualmente, una capa de políticas que se sitúa *entre* la aplicación y el modelo fundacional (FM), y que evalúa tanto lo que entra (el prompt del usuario) como lo que sale (la respuesta generada) antes de que llegue a su destino final. Su característica más importante — y una de las más preguntadas en el examen — es que es **independiente del modelo**: la misma guardrail se puede asociar a cualquier modelo servido por Amazon Bedrock, a un Amazon Bedrock Agent, a una Knowledge Base, o incluso invocarse contra las respuestas de un modelo alojado *fuera* de AWS (en tu propia infraestructura o en otro proveedor), mediante la API independiente `ApplyGuardrail`.

Esta independencia tiene una consecuencia práctica que aparece repetidamente en escenarios de examen: `ApplyGuardrail` está **desacoplada de la invocación del modelo** (“decoupled from foundation models”). Esto significa que puedes evaluar un guardrail contra un texto en cualquier punto de tu flujo de aplicación —antes de llamar al modelo, después de recibir la respuesta, o incluso sobre contenido que nunca pasó por un FM de Bedrock— sin que eso dispare una inferencia. Cuando un enunciado describe la necesidad de aplicar distintas políticas de moderación según el grupo de usuario (por ejemplo, mostrar PII completa a cirujanos pero enmascararla para ingenieros, sobre la misma respuesta de un agente), la pista casi siempre apunta a llamar

dinámicamente a `ApplyGuardrail` con el `guardrailIdentifier` correspondiente al rol del usuario autenticado, en lugar de duplicar infraestructura o bases de conocimiento.

Una guardrail no es una política única, sino un conjunto de políticas combinables que se configuran juntas en un mismo recurso: **denied topics**, **content filters**, **sensitive information filters**, **word filters** y **contextual grounding check**. El examen penaliza sistemáticamente las soluciones que fragmentan estas políticas en múltiples guardrails encadenadas mediante Step Functions u orquestación propia cuando una única guardrail bien configurada ya cubre todos los casos — esto añade complejidad operativa sin ningún beneficio funcional.

4.2 Denied Topics: bloqueo semántico de intenciones

Los **Denied Topics** no funcionan por coincidencia de palabras, sino por comprensión semántica: se define un nombre de tema, una **definición clara y sin ambigüedad** (hasta 200 caracteres) y hasta cinco frases de ejemplo que ilustran el tipo de conversación que se quiere bloquear. La guardrail utiliza esa definición para reconocer variantes, sinónimos y formulaciones distintas del mismo tema, no solo las frases exactas proporcionadas.

Esto es clave para distinguir Denied Topics de los Content Filters: si el requisito es impedir que un asistente financiero “recomiende acciones concretas” o “prometa retornos garantizados”, ningún Content Filter puede capturar esa intención de negocio porque los Content Filters solo entienden categorías de daño genérico (odio, violencia, etc.), no dominios de negocio específicos. La AWS recomienda explícitamente definiciones precisas y sin ambigüedad, porque una definición vaga reduce la exactitud de la detección semántica.

4.3 Content Filters: las seis categorías de daño y sus niveles de confianza

Los **Content Filters** evalúan texto (y, en modelos que lo soportan, también imágenes) contra seis categorías predefinidas: **Hate** (odio/discriminación por identidad), **Insults** (lenguaje humillante o denigrante — la documentación aclara explícitamente que este tipo de lenguaje también se etiqueta como *bullying*), **Sexual**, **Violence**, **Misconduct** (mala conducta / actividades poco éticas o ilegales) y **Prompt Attack** (jailbreaks, inyección de prompt y fuga de instrucciones del sistema).

Un matiz que el examen exige dominar con precisión es cómo funciona la escala de severidad (**None**, **Low**, **Medium**, **High**), que se configura de forma independiente para el tráfico de entrada y de salida: cada categoría produce internamente una clasificación de confianza (NONE, LOW, MEDIUM, HIGH) sobre si el contenido pertenece a esa categoría de daño, y el nivel de severidad que configuras determina **a partir de qué nivel de confianza se bloquea el contenido**:

- **High**: bloquea contenido clasificado con confianza HIGH, MEDIUM y LOW. Solo deja pasar confianza NONE. Es la configuración más agresiva y, por tanto, la que genera más falsos positivos.
- **Medium**: bloquea únicamente confianza HIGH y MEDIUM, dejando pasar LOW y NONE. Es el punto intermedio entre eficacia de filtrado y minimización de falsos positivos.
- **Low**: bloquea solo confianza HIGH.

Cuando un enunciado pide explícitamente “minimizar falsos positivos” sin dejar de filtrar contenido claramente dañino, la respuesta correcta casi nunca es “High para todas las categorías” (eso maximiza los falsos positivos), sino **Medium**, salvo que el enunciado exija la posición más estricta posible sin importar el ruido. Para ataques

de prompt sofisticados, en cambio, sí se busca típicamente la severidad **High**, porque el coste de dejar pasar un jailbreak suele ser mayor que el de un falso positivo ocasional.

Los Content Filters también soportan un modo **Detect** (además de **Block**): en modo Detect, la guardrail identifica y registra el contenido dañino en la traza para su seguimiento, pero **no bloquea** la respuesta. Este matiz aparece explícitamente en escenarios donde la empresa quiere ser *notificada* de lenguaje de acoso o bullying sin bloquear automáticamente todas las respuestas marcadas: la combinación correcta es la categoría **Insults** con acción **Detect**, no **Block**.

4.4 Sensitive Information Filters (PII): Mask vs Block, y el matiz de residencia de datos

Los **Sensitive Information Filters** detectan información de identificación personal —nombres, direcciones, números de tarjeta, números de la seguridad social, correos electrónicos, teléfonos— tanto mediante tipos de entidad PII predefinidos como mediante **expresiones regulares personalizadas** para formatos propios de la empresa (números de póliza, identificadores internos, etc.). La política admite configurar la acción de forma general y también de forma diferenciada por dirección del tráfico (acción específica de entrada frente a acción específica de salida), lo que permite, por ejemplo, **bloquear** información financiera sensible en las preguntas del usuario mientras se **enmascara** (no se bloquea) la PII que aparece en las respuestas del modelo, dentro de la misma guardrail.

La distinción entre las dos acciones es uno de los “gotchas” más repetidos del examen:

- **Mask (ANONYMIZE)**: sustituye el dato sensible detectado por una etiqueta genérica (por ejemplo `{NAME}`, `{EMAIL}`) pero **deja pasar el resto de la respuesta**. Es la opción correcta cuando el requisito es que la conversación *continúe* con normalidad sin exponer el dato real — y también es la opción recomendada durante fases de desarrollo y pruebas, porque permite al equipo ver exactamente qué está detectando el guardrail como PII sin perder el resto del contexto de la respuesta.
- **Block**: descarta la respuesta completa en cuanto detecta cualquier coincidencia de PII. Es la opción correcta para el entorno de producción cuando el requisito es una garantía absoluta de que ningún dato personal llegue nunca al usuario final.

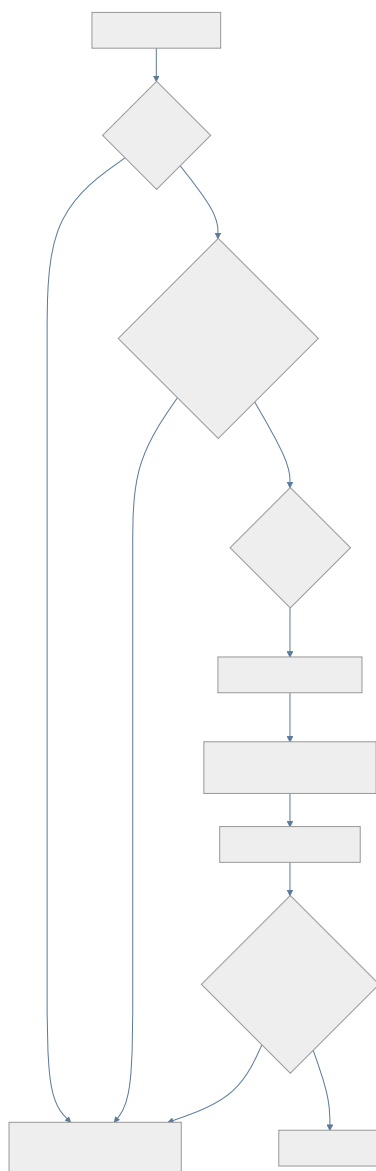
Un patrón de examen recurrente es: *detect/mask en desarrollo* → *block en producción*, sobre el mismo guardrail, ajustando solo el modo cuando la aplicación pasa de pruebas a producción real.

Aquí aparece también un matiz importante sobre el **guardrail profile** (el mecanismo de inferencia cross-Region de Guardrails, análogo a los Cross-Region Inference Profiles de los modelos): aunque la configuración de la guardrail reside en tu Región principal, al asociar un guardrail profile las solicitudes de evaluación de política pueden enrutarse y procesarse en otras Regiones dentro de la misma geografía. Si el enunciado exige que **todo el procesamiento ocurra exactamente en la Región donde está el cliente** (residencia de datos estricta a nivel de Región, no de geografía), un guardrail cross-Region **incumple** ese requisito, y la solución correcta pasa por desplegar una **copia independiente de la guardrail en cada Región** en la que opera la empresa, en lugar de usar un guardrail profile.

4.5 Word Filters y listas de bloqueo por coincidencia exacta

Los **Word Filters** bloquean palabras y frases mediante **coincidencia léxica exacta**, tanto en prompts de entrada como en respuestas del modelo. Se pueden cargar listas personalizadas (hasta 10.000 elementos) y la documentación de AWS menciona explícitamente su uso para bloquear nombres de competidores o de productos de la competencia — un caso que ningún Content Filter puede resolver, porque los Content Filters trabajan sobre categorías semánticas de daño, no sobre términos léxicos concretos definidos por el negocio. Cuando el enunciado pide impedir que el asistente mencione marcas específicas, la respuesta casi siempre combina **Word Filters** (para los nombres exactos) con **Denied Topics** (para los patrones de conversación de riesgo de negocio más amplios), no Content Filters.

4.6 Contextual Grounding Check: la defensa nativa contra alucinaciones en RAG



Flujo de evaluación de entrada/salida en Amazon Bedrock Guardrails

El **Contextual Grounding Check** es el mecanismo diseñado específicamente para detectar y filtrar alucinaciones en escenarios de resumen, paráfrasis y respuesta a preguntas (no está soportado, según la documentación oficial, en chat conversacional abierto). Requiere tres componentes: una **fuentes de referencia** (grounding source, hasta 100.000 caracteres — por ejemplo, los fragmentos recuperados de una Knowledge Base), la **consulta del usuario** (hasta 1.000 caracteres) y el **contenido a evaluar** (la respuesta del modelo, hasta 5.000 caracteres).

Genera dos puntuaciones de confianza independientes:

- **Grounding:** si la respuesta es factualmente correcta y se deduce de la fuente de referencia. Cualquier información nueva no presente en la fuente se considera “no fundamentada”, incluso si es objetivamente cierta.
- **Relevance:** si la respuesta atiende realmente a la pregunta formulada, con independencia de si está bien fundamentada en la fuente.

Un ejemplo oficial ilustra bien la diferencia: si la fuente dice “Londres es la capital del Reino Unido. Tokio es la capital de Japón” y la pregunta es “¿Cuál es la capital de Japón?”, una respuesta como “La capital de Japón es Londres” es **relevante pero no fundamentada** (mezcla datos de la fuente de forma incorrecta), mientras que “La capital del Reino Unido es Londres” es **fundamentada pero no relevante** (correcta, pero no responde a la pregunta).

Corrección importante sobre la guía original: el umbral de Grounding y Relevance no se configura en un rango de 0.0 a 1.0, sino específicamente entre **0 y 0.99** — un valor de 1 no es válido porque bloquearía la totalidad del contenido, dejando el guardrail inservible. Cuanto más alto el umbral, mayor la probabilidad de bloquear contenido no fundamentado o irrelevante, a costa de más falsos positivos; este es precisamente el mecanismo que hay que subir cuando el requisito es que las respuestas se ciñan estrictamente a una guía o fuente aprobada (por ejemplo, en un asistente financiero regulado), y es una defensa mucho más fiable que “bajar la temperatura”, que reduce la aleatoriedad del modelo pero no garantiza en absoluto que el contenido esté anclado en una fuente real.

4.7 Aplicación práctica: versionado, `ApplyGuardrail` y observabilidad

Cada guardrail se puede versionar (borrador editable + versiones numeradas inmutables publicadas), y se referencia en las llamadas de inferencia mediante `guardrailIdentifier` y `guardrailVersion`. La API `ApplyGuardrail`, al ser independiente del modelo, permite evaluar un guardrail concreto sobre cualquier texto sin invocar ningún FM — esto es lo que hace posible seleccionar dinámicamente, en tiempo de ejecución y según el rol del usuario autenticado (por ejemplo, vía grupos de Amazon Cognito), qué guardrail aplicar sobre la misma respuesta generada por un agente o Knowledge Base compartida, evitando así duplicar bases de conocimiento o pipelines de datos completos solo para servir distintos niveles de exposición de PII.

Para auditoría, Amazon Bedrock Guardrails publica en CloudWatch la métrica `InvocationsIntervened`, que se puede desglosar con varias dimensiones nativas — no solo `GuardrailContentSource` (INPUT u OUTPUT), sino también `Operation`, `GuardrailPolicyType` y `GuardrailArn` — lo que permite construir alarmas y dashboards granulares por tipo de política que intervino, sin necesidad de instrumentación personalizada. Además, habilitar el *model invocation logging* (con destino en Amazon S3 o CloudWatch Logs) permite registrar, junto con las trazas de guardrail, el contenido completo de las interacciones — incluidas imágenes o

documentos transmitidos, si se activa explícitamente la entrega (“delivery logging”) y el registro de imágenes (“image logging”) en S3 — cubriendo así los requisitos de pistas de auditoría para cumplimiento normativo.

Caso de Estudio — Asesor financiero con múltiples riesgos

Una entidad financiera necesita impedir que su asistente dé consejos de inversión concretos, evitar que mencione a la competencia, y garantizar que sus respuestas no contengan afirmaciones no respaldadas por la documentación financiera aprobada de la empresa. La combinación ganadora es: **Denied Topics** para los patrones de conversación de alto riesgo (“recomendaciones específicas de inversión”, “promesas de retorno garantizado”), **Word Filters** con acción de bloqueo para los nombres exactos de la competencia, y un **umbral alto de Grounding Score** en el Contextual Grounding Check. Un Content Filter no sirve para ninguno de los dos primeros casos, porque trabaja sobre categorías de daño genérico, no sobre intenciones de negocio ni nombres propios; y un umbral *bajo* de grounding dejaría pasar precisamente las afirmaciones no ancladas en la fuente que la empresa quiere evitar.

Caso de Estudio — Notificar sin bloquear

Un asistente basado en Amazon Nova Pro, orquestado mediante Amazon Bedrock Flows, debe generar una notificación cuando una respuesta contenga lenguaje de acoso, pero sin bloquear automáticamente esas respuestas. La categoría de Content Filter correcta es **Insults** (que la documentación asocia explícitamente con *bullying*), configurada con acción **Detect** en lugar de **Block** — Detect registra la intervención para su seguimiento sin impedir que la respuesta llegue al usuario. La categoría **Hate** queda descartada porque cubre discriminación por identidad, no acoso en general; y configurar la acción como Block contradiría el requisito explícito de no bloquear todas las respuestas marcadas.

Caso de Estudio — PII condicionada por rol clínico

Un fabricante de dispositivos médicos necesita que los cirujanos vean la PII de los pacientes en los informes, pero que los ingenieros solo vean versiones redactadas, sobre la misma Knowledge Base y sin duplicar datos. La solución nativa es invocar la API **ApplyGuardrail** — desacoplada de la invocación del modelo — seleccionando dinámicamente el `guardrailIdentifier` correspondiente al grupo de Amazon Cognito del usuario autenticado (un guardrail que enmascara PII para ingenieros, otro que la deja pasar para cirujanos), en lugar de mantener dos Knowledge Bases separadas o redactar los documentos de forma permanente en el origen, lo que destruiría la PII también para quienes sí deben verla.

Fuentes

- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-denied-topics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-content-filters-overview.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-word-filters.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>

- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-prompt-attack.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-cross-region.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-use-independent-api.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-guardrails-cw-metrics.html>
 - <https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html>
 - https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ApplyGuardrail.html ##
- MÓDULO 5: MONITORIZACIÓN, TRAZABILIDAD Y EVALUACIÓN DE MODELOS

La monitorización de aplicaciones de IA generativa tiene una particularidad que la distingue de la observabilidad tradicional de software: no basta con saber que un servicio responde rápido y sin errores HTTP; hay que saber si lo que responde es **correcto, fiel a sus fuentes y justo**. Este módulo cubre las dos caras de esa moneda —evaluación de calidad y observabilidad operativa— y es, junto con Seguridad y RAG, uno de los bloques con más peso real en el examen. La clave para superar sus preguntas no es memorizar servicios sueltos, sino entender qué mide exactamente cada herramienta y, sobre todo, qué NO mide, porque casi todos los distractores del examen consisten en aplicar la herramienta correcta a la métrica equivocada, o una herramienta reactiva donde se pedía algo proactivo/automático.

5.1 Model Evaluation en Amazon Bedrock: automática vs. humana

Amazon Bedrock Evaluations permite comparar el rendimiento de distintos FMs (o distintas variantes de prompt) frente a un conjunto de datos de entrada, sin necesidad de construir un pipeline de evaluación propio. Existen dos modalidades:

- **Automatic Evaluation:** aplica métricas algorítmicas y de referencia (Exact Match, F1, ROUGE/BLEU, Toxicity, Accuracy, Robustness) o, de forma más moderna, un **modelo evaluador (“LLM-as-a-judge”)** — una versión soportada de Anthropic Claude, Amazon Nova u otro modelo autorizado como juez— que puntúa las respuestas y devuelve tanto la puntuación como una explicación textual del porqué. Esta modalidad es la respuesta correcta siempre que el enunciado pida objetividad, reproducibilidad y **procesamiento a escala sin intervención manual**.
- **Human Evaluation:** se apoya en **SageMaker Ground Truth Workforces**, con tres orígenes de anotadores posibles: **Private Team** (empleados propios, para datos sensibles o dominios muy especializados), **Vendor** (proveedores certificados por AWS) y **Mechanical Turk** (multitud pública, solo para tareas no sensibles). Es la respuesta correcta cuando el criterio a evaluar es inherentemente subjetivo: tono de marca, adecuación cultural, creatividad, o calidad de un resumen que ningún score automático puede capturar bien.

Un matiz que el examen explota con frecuencia: **combinar ambas modalidades no es redundante, es la mejor práctica**. Un patrón recurrente en los escenarios reales es un sistema híbrido donde el LLM-as-a-judge criba automáticamente el grueso del tráfico y solo los casos marcados como límite o de alto riesgo (por ejemplo, en un dominio clínico o financiero) pasen a revisión humana dirigida —normalmente mediante **Amazon Augmented AI (Amazon A2I)**, que permite configurar “human review workflows” que se activan solo ante ciertas condiciones—. Esta arquitectura reduce drásticamente el coste de revisión manual sin renunciar a una red de seguridad humana en los casos que importan.

Bedrock también ofrece **evaluaciones específicas para RAG**, con dos tipos de trabajo bien diferenciados que el examen distingue con precisión quirúrgica:

- **Retrieve-only:** evalúa exclusivamente la calidad de la recuperación de la base de conocimiento, con métricas integradas como *Context Relevance* (¿los fragmentos recuperados son relevantes?) y *Context Coverage*. No invoca ni evalúa el modelo generador.
- **Retrieve-and-generate:** evalúa el sistema completo, incluida la respuesta generada, con la métrica *Builtin.Faithfulness* (mide explícitamente si la respuesta evita alucinar respecto a los pasajes recuperados) y métricas de comparación con una respuesta de referencia como *Builtin.Correctness* y *Builtin.Relevance*. Si el enunciado exige comparar la **calidad de generación** entre dos FMs o dos estrategias de chunking, la opción correcta siempre pasa por un trabajo *retrieve-and-generate*, nunca por uno *retrieve-only*. Además, cada trabajo de evaluación RAG de Bedrock se define contra una única fuente RAG (`knowledgeBaseId` o `ragSourceIdentifier`) y un único modelo generador, así que comparar varias estrategias de chunking o varios FMs a la vez implica en la práctica ejecutar **varios trabajos** y comparar después sus resultados.

Para el acceso a datos, hay un detalle que aparece explícitamente en el examen: el bucket de S3 con los datasets de entrada solo necesita permisos de IAM adecuados para el rol de servicio de Bedrock; **la configuración de CORS solo es obligatoria en evaluaciones con revisión humana** (para que el portal de anotadores pueda renderizar los prompts en el navegador), no en evaluaciones automáticas. Añadir un VPC endpoint tampoco es necesario salvo que el tráfico deba permanecer dentro de una VPC sin salida a Internet.

5.2 Framework RAGAS y evaluación LLM-as-a-Judge en producción

Cuando un sistema RAG ya está en producción y se quiere medir su calidad de forma continua (no solo en un trabajo de evaluación puntual), la industria converge en el framework conceptual **RAGAS** (*Retrieval Augmented Generation Assessment*), que descompone la calidad en tres ejes:

Calidad RAG = $f(\text{Context Relevance}, \text{Faithfulness}, \text{Answer Relevance})$

1. **Context Relevance:** ¿los fragmentos que trajo la base vectorial son relevantes para la pregunta, sin ruido?
2. **Faithfulness / Groundedness:** ¿todas las afirmaciones de la respuesta se pueden deducir del contexto recuperado? Es la métrica anti-alucinación por excelencia.
3. **Answer Relevance:** ¿la respuesta atiende realmente a la pregunta formulada, con independencia de si el contexto era completo?

Estos mismos tres ejes son, en esencia, los que Bedrock Evaluations materializa con sus métricas integradas (Context Relevance/Coverage, Faithfulness, Correctness/Relevance), y también los que se pueden replicar con métricas personalizadas: Bedrock permite definir hasta 10 métricas por trabajo de evaluación, cada una con su propio prompt de instrucciones para el modelo juez y su propio esquema de puntuación (por ejemplo, una escala de 1 a 5), lo que permite construir métricas a medida como “precision at k” para escenarios muy específicos, como comparar múltiples estrategias de chunking entre sí.

5.3 Amazon SageMaker Clarify: sesgo, equidad y deriva

SageMaker Clarify es el servicio de AWS diseñado específicamente para el análisis de **sesgo (bias)** y **equidad (fairness)**, y es un tema que reaparece en el examen con enunciados que, a primera vista, parecen de observabilidad pero en realidad piden equidad estadística. La señal inequívoca es cualquier mención a “grupos demográficos”, “disparidad entre segmentos” o “favorecido/desfavorecido”: ahí, la respuesta casi siempre es Clarify, no Guardrails ni CloudWatch a secas.

Clarify distingue dos momentos de análisis. Las **métricas pre-entrenamiento** se calculan sobre el dataset crudo, antes de entrenar o invocar ningún modelo, y responden a la pregunta “¿los propios datos ya están desequilibrados entre grupos?”; ejemplos típicos son el **Class Imbalance (CI)** (desequilibrio en el tamaño de cada facet) o la **Difference in Proportions of Labels (DPL)** (diferencia en la proporción de resultados positivos observados entre el grupo favorecido y el desfavorecido). Las **métricas post-entrenamiento** —hasta **once** métricas distintas— se calculan sobre las predicciones del modelo ya entrenado (o sobre las respuestas de un modelo generativo) y cuantifican si el sesgo se mantuvo, se corrigió o se amplificó; las más citadas en el examen son la **Difference in Positive Proportions in Predicted Labels (DPPL)**, el **Disparate Impact (DI)** — el cociente, no la diferencia, entre la proporción de resultados positivos del grupo desfavorecido y el favorecido — y la **Accuracy Difference (AD)**, que compara la precisión del modelo entre ambos grupos. Todas estas métricas comparan resultados entre “**facets**”, es decir, los grupos demográficos favorecido y desfavorecido que el analista define explícitamente (por ejemplo, edad, género o código postal), y esta misma lógica de comparación por facets se extiende también a la evaluación de modelos de lenguaje o generativos, no solo a clasificadores clásicos.

Un punto muy explotado en el examen: los trabajos de **monitorización de deriva de sesgo (bias drift monitoring)** de Clarify —ejecutados como SageMaker Model Monitor jobs sobre un endpoint o sobre un batch transform— publican estas métricas automáticamente en Amazon CloudWatch, bajo el namespace `aws/sagemaker/Endpoints/bias-metrics` para endpoints en tiempo real, o `aws/sagemaker/ModelMonitoring/bias-metrics` para trabajos por lotes (batch transform). Cada métrica se publica con propiedades/dimensiones que incluyen `Endpoint`, `MonitoringSchedule`, `BiasStage` (con valor `Pre-training` o `Post-Training`, para distinguir de qué fase procede el dato), `Label / LabelValue` (la etiqueta objetivo) y, sobre todo, `Facet` y `FacetValue`, que identifican exactamente qué grupo demográfico está detrás de cada punto de la serie temporal. Esto permite crear alarmas de CloudWatch —incluidas **alarmas compuestas** que combinen estas métricas de sesgo con métricas de latencia u otras métricas de negocio de Bedrock— para alertar ante una discrepancia entre grupos que supere un umbral (por ejemplo, un 15%), y generar dashboards y reportes periódicos comparando variantes de prompt o de modelo. Conviene una precisión honesta de cara al examen: tanto SageMaker Clarify como el propio SageMaker Model Monitor han pasado a un estado de disponibilidad limitada para clientes nuevos (los clientes existentes siguen operando con normalidad y AWS mantiene el servicio), pero eso no cambia el hecho de que, entre las opciones típicas de un enunciado, sigue siendo el único servicio de AWS con métricas de equidad estadística nativas y publicación automática en CloudWatch.

Es fundamental no confundir esto con las métricas de **Amazon Bedrock Guardrails**, publicadas bajo el namespace `AWS/Bedrock/Guardrails`: sus dimensiones nativas en CloudWatch son `Operation`, `GuardrailContentSource` (valores `Input / Output`), `GuardrailPolicyType` (valores `ContentPolicy`, `TopicPolicy`, `WordPolicy`, `SensitiveInformationPolicy`, `ContextualGroundingPolicy`) y `GuardrailArn / GuardrailVersion` —no existe ninguna dimensión “por grupo demográfico”—, y su métrica

`InvocationsIntervened` cuenta cuántas invocaciones fueron bloqueadas o modificadas por el guardrail, no calcula ninguna disparidad estadística entre segmentos de población. Un guardrail con filtros de contenido detecta odio, insultos o contenido sexual; no mide equidad.

5.4 Bedrock Trace Events: depurando el razonamiento de un agente

Cuando el objeto a monitorizar no es un modelo aislado sino un **Bedrock Agent**, la trazabilidad relevante no vive en métricas de CloudWatch sino en los **eventos de traza** que el propio servicio emite en cada invocación, divididos en cuatro fases:

1. **PreProcessingTrace**: clasificación de la intención del usuario y validación de la entrada.
2. **OrchestrationTrace**: el razonamiento paso a paso (ReAct), qué herramienta o Knowledge Base decidió invocar el agente y con qué parámetros.
3. **PostProcessingTrace**: formateo de la respuesta final antes de devolverla al usuario.
4. **FailureTrace**: diagnóstico de fallos concretos, típicamente invocaciones de Lambda que fallan o esquemas OpenAPI de un Action Group que no validan.

Cuando un escenario de examen describe un agente que “falla al llamar a una API” o “no ejecuta la acción esperada”, la fuente de diagnóstico correcta es la traza de orquestación/fallo del propio agente, no una métrica genérica de CloudWatch ni un log de aplicación construido a medida.

5.5 Model Invocation Logging y observabilidad de tokens

Model Invocation Logging es la funcionalidad nativa que registra, sin escribir una sola línea de código, el texto completo de los prompts y las respuestas para las operaciones `InvokeModel`, `InvokeModelWithResponseStream`, `Converse` y `ConverseStream`, junto con metadatos de cada llamada (identidad del invocador, modelo, tokens, latencia) y, cuando corresponde, los propios **vectores de embeddings** generados por un modelo de embeddings. Cuando se usa la API `Converse` con contenido multimodal, las imágenes o documentos adjuntos también quedan registrados en el bucket de S3, siempre que la entrega y el registro de imágenes estén habilitados. Los dos destinos posibles son un bucket de **Amazon S3** o un grupo de logs de **Amazon CloudWatch Logs** —ambos deben residir en la misma cuenta y región que la configuración de logging—, y ambos destinos admiten cifrado en reposo con **AWS KMS** (incluida una política de KMS específica para que el servicio de Bedrock pueda generar claves de datos). Sobre estos logs es donde el examen construye buena parte de sus escenarios de “consumo anómalo de tokens”:

- Amazon Bedrock publica de forma nativa en CloudWatch, bajo el namespace `AWS/Bedrock`, métricas de runtime como `Invocations`, `InputTokenCount` y `OutputTokenCount`, listas para dashboards y alarmas sin ETL adicional.
- Cuando se necesita **atribuir** el consumo a una aplicación, equipo o, sobre todo, a una **herramienta específica dentro de un agente**, la vía nativa es enviar el invocation logging a **CloudWatch Logs** y crear **metric filters** que extraigan patrones concretos (por ejemplo, filtrando por el nombre de la tool invocada) y los publiquen como métricas personalizadas por herramienta.
- Cuando el requisito añade que los **umbrales deben adaptarse automáticamente** a medida que cambia el tráfico —en vez de alarmas con umbrales fijos que hay que revisar a mano—, la pieza que cierra el

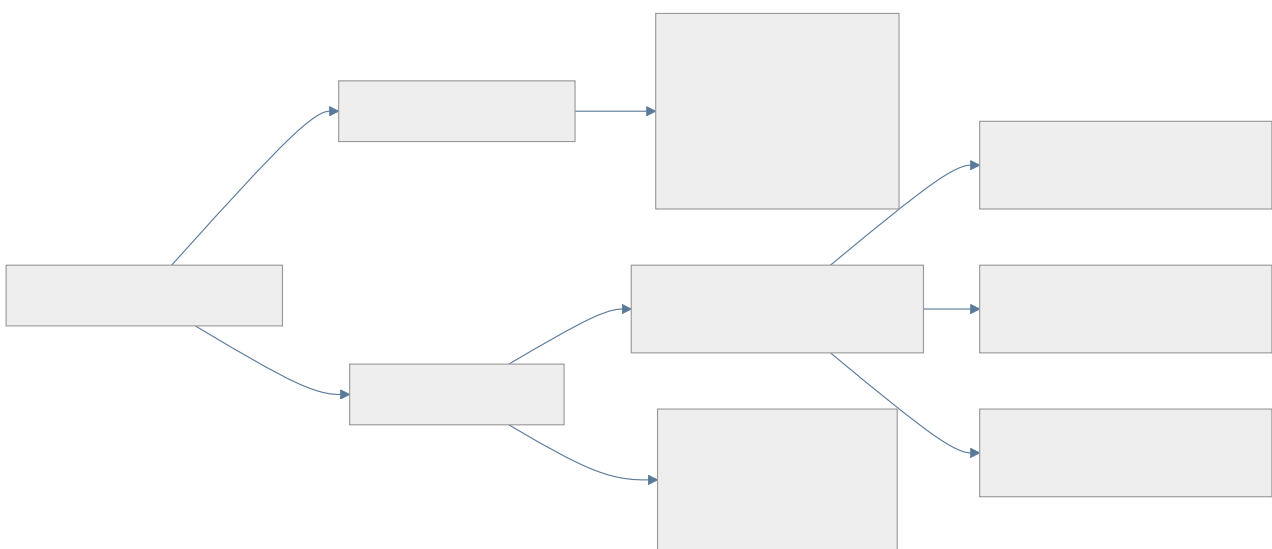
escenario es **CloudWatch Anomaly Detection**: aplica modelos estadísticos que aprenden una “banda” de comportamiento normal por métrica (con estacionalidad horaria/diaria/semanal) y disparan la alarma cuando el valor real se sale de esa banda, sin intervención manual.

- Para estimar el consumo de tokens **antes** de invocar el modelo (por ejemplo, para alertas proactivas de cuota por unidad de negocio a gran escala), Bedrock ofrece la API nativa **CountTokens**, sin coste adicional, como alternativa moderna a construir tokenizadores propios por modelo.

5.6 Contextual Grounding Check: el detector de alucinaciones en tiempo real

Conviene remarcar una distinción que el examen convierte en trampa recurrente: **Bedrock Model Evaluation es, por diseño, un proceso por lotes/offline** —se ejecuta contra un dataset de prompts antes o durante el desarrollo, no intercepta tráfico en producción—. El mecanismo que sí actúa **en el momento de cada invocación real** (a través de `InvokeModel`, `Converse` o `ApplyGuardrail`) para detectar y **bloquear** alucinaciones es la **Contextual Grounding Check** dentro de Amazon Bedrock Guardrails, que asigna a cada respuesta una puntuación de *grounding* (fidelidad a la fuente) y otra de *relevance*, con umbrales configurables. Si un escenario pide detección de alucinaciones “casi en tiempo real” o “antes de que la respuesta llegue al usuario”, la respuesta correcta pasa por Guardrails, no por un trabajo de Model Evaluation.

Para la resiliencia operativa frente a fallos transitorios de la API (throttling, timeouts), el examen premia usar los **modos de reintento estándar del AWS SDK** (`standard`, con backoff exponencial y jitter incorporados, y un “token bucket” de cuota de reintentos que evita seguir reintentando ante fallos generalizados) combinados con **AWS X-Ray** para trazado distribuido real entre los límites de servicio, incluidas anotaciones indexables para correlacionar la latencia con un modelo o versión concreta. El modo `adaptive` existe y es válido, pero AWS lo recomienda solo para el caso concreto de un cliente apuntando a un único recurso con throttling frecuente, no como opción general; y **AWS CloudTrail no traza solicitudes distribuidas**, solo audita quién invocó qué API y cuándo.



Evaluación automática vs. humana en Amazon Bedrock

Caso de Estudio — Sesgo demográfico en un sistema de recomendación

Una empresa de retail necesita detectar disparidades de más del 15% entre grupos demográficos en las recomendaciones de dos variantes de prompt, con alertas casi en tiempo real y reportes semanales. La tentación es pensar en Guardrails (por la palabra “contenido”) o en un pipeline de EventBridge + Lambda a medida. Ambas fallan: los content filters de Guardrails no miden disparidad estadística y no existe una dimensión por grupo demográfico en sus métricas de CloudWatch, y construir la lógica de equidad desde cero contradice el requisito de mínimo esfuerzo de desarrollo. La respuesta ganadora combina **SageMaker Clarify** (que sí calcula métricas de equidad entre facets) publicando en CloudWatch, con alarmas compuestas que correlacionan esas métricas de sesgo con métricas de latencia de Bedrock.

Caso de Estudio — Consumo de tokens que varía por herramienta de un agente

Una aplicación con integraciones de herramientas sufre picos de consumo de tokens inexplicables pese a un tráfico de usuario estable, y se pide identificar qué herramienta concreta lo causa, con umbrales que se adapten solos al tráfico. Un dashboard con alarmas de umbral fijo (por herramienta) resuelve la visibilidad pero no la adaptación automática; consultas programadas por lotes sobre S3 con Athena o Glue añaden retraso y mantenimiento. La combinación ganadora es enviar el invocation logging a **CloudWatch Logs**, extraer patrones por herramienta con **metric filters**, y aplicar **CloudWatch Anomaly Detection** sobre esas métricas por herramienta: el modelo estadístico de banda esperada se recalcula solo con el histórico, sin tocar un umbral a mano.

Caso de Estudio — Gobernanza integral con plazo y latencia ajustados

Una entidad financiera necesita, en 60 días y con menos de 200 ms de latencia añadida, detectar alucinaciones, aplicar controles de seguridad, monitorizar deriva y mantener un audit trail para regulación, integrándose con un dashboard de cumplimiento ya existente. Las opciones que introducen infraestructura propia de bases de datos relacionales o de búsqueda (RDS, OpenSearch) o servicios de analítica adicionales (QuickSight, SageMaker Model Monitor fuera de Bedrock) multiplican el trabajo de aprovisionamiento y mantenimiento. La combinación con menor sobrecarga operativa apoya los controles de contenido en **Guardrails**, el audit trail en **DynamoDB** (con TTL, sin gestión de clústeres) y la integración de métricas en **CloudWatch personalizado**: tres servicios serverless que cumplen el plazo y la latencia sin construir infraestructura dedicada.

Caso de Estudio — Puerta de calidad multilingüe en el pipeline de CI/CD

Tras actualizar el modelo que sostiene un asistente multilingüe de atención al cliente, una empresa observa que el asistente empieza a responder de forma distinta ante preguntas equivalentes según el idioma. Para futuras actualizaciones, necesita evaluar más de 15.000 conversaciones de prueba en paralelo, en menos de 45 minutos, de forma completamente automatizada e integrada en el pipeline de CI/CD, bloqueando el despliegue si la calidad no alcanza el umbral exigido. Enfoques centrados solo en infraestructura (simulación de tráfico y métricas de latencia/concurrencia) o en desplegar en más regiones con auditorías manuales semanales no evalúan en ningún momento si el *significado* de la respuesta se mantiene entre idiomas, y tampoco pueden bloquear un despliegue de forma automática. La solución correcta construye conjuntos de conversaciones de prueba estandarizadas con significado idéntico en cada idioma soportado y los ejecuta como **trabajos de evaluación de modelos de Amazon Bedrock** en paralelo, aplicando métricas integradas como *Faithfulness* (alucinación) y *Correctness*/similitud semántica frente a una respuesta de referencia; el pipeline de CI/CD consulta el resultado del trabajo vía API y bloquea la promoción a producción si los umbrales de similitud o alucinación no se cumplen, cerrando así el ciclo de “evaluación como puerta de calidad” en lugar de como auditoría posterior.

Fuentes

- <https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-measure-post-training-bias.html>
- <https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-model-monitor-bias-drift-cw.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-guardrails-cw-metrics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-evaluation-metrics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-judge.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-kb.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-built-in-metrics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-type-automatic.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/count-tokens.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-how.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/trace-events.html>
- https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Anomaly_Detection.html
- <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/appinsights-what-is.html>
- <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/alarm-combining.html>
- <https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html>
- <https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html>
- <https://docs.aws.amazon.com/sagemaker/latest/dg/a2i-use-augmented-ai-a2i-human-review-loops.html>

- <https://docs.aws.amazon.com/wellarchitected/latest/agentic-ai-lens/agentops02-bp01.html> ## MÓDULO 6: RENDIMIENTO, ESCALABILIDAD Y OPTIMIZACIÓN DE COSTES

En una aplicación de IA Generativa, el “rendimiento” no es una única variable: es la combinación de la latencia percibida por el usuario (cuánto tarda en aparecer el primer carácter de la respuesta), el throughput sostenido que la arquitectura puede absorber sin degradarse, y el coste por unidad de trabajo (por token, por invocación, por hora de capacidad reservada). Este módulo profundiza en las tres palancas que Amazon Bedrock ofrece para mover esas variables — streaming, prompt caching y los distintos modos de inferencia — y en cómo diseñar una aplicación resiliente frente al throttling, que es la forma en que Bedrock comunica que se ha superado una cuota de servicio.

6.1 Latencia percibida: streaming de respuestas

Cuando un modelo fundacional genera una respuesta larga, el tiempo total de generación puede oscilar entre varios segundos y más de un minuto. Si la aplicación espera a tener el texto completo antes de mostrar nada, el usuario percibe ese tiempo total como “lentitud” o incluso como un fallo (timeout). La técnica que resuelve esto no es acelerar al modelo, sino cambiar el contrato de entrega: en lugar de una única respuesta al final, el servicio entrega la respuesta en fragmentos (*chunks*) a medida que el modelo los genera. La métrica que de verdad le importa al usuario deja de ser la latencia total y pasa a ser el **Time To First Token (TTFT)**: el tiempo hasta que aparece el primer fragmento visible.

Amazon Bedrock expone esta capacidad a través de las variantes de streaming de sus dos familias de API: `ConverseStream` (recomendada, formato unificado entre proveedores) e `InvokeModelWithResponseStream` (payload específico del proveedor). Para flujos RAG existe además `RetrieveAndGenerateStream`, que combina recuperación y generación incremental en una sola llamada — una API distinta de `InvokeModelWithResponseStream`, que solo invoca el modelo y no ejecuta ninguna lógica de recuperación contra una Knowledge Base. Confundir ambas es un error de diseño frecuente: sustituir `RetrieveAndGenerate` / `RetrieveAndGenerateStream` por `InvokeModelWithResponseStream` obliga a reimplementar manualmente toda la orquestación de recuperación que Bedrock ya gestiona.

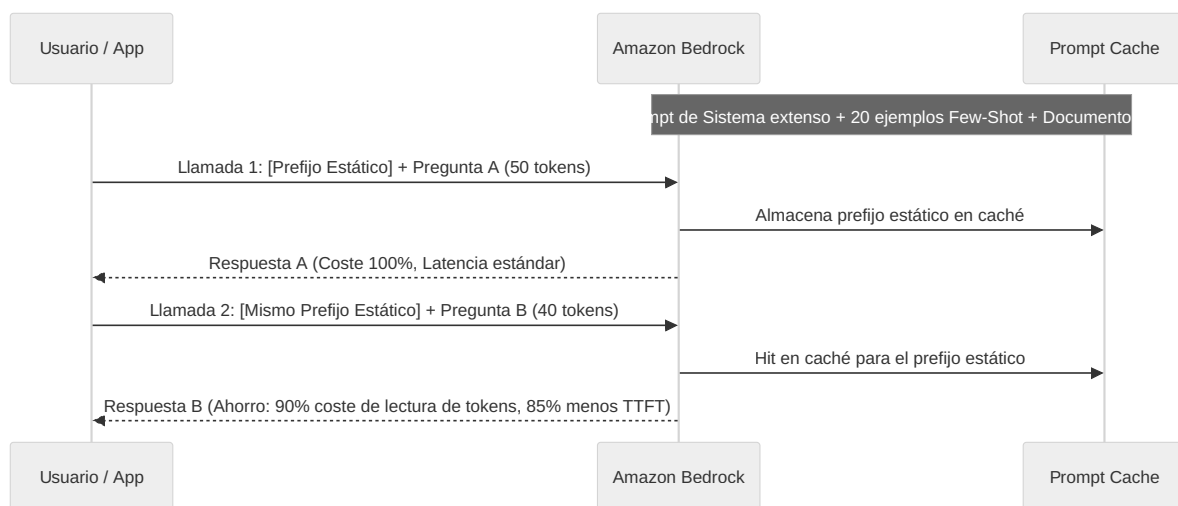
Recibir los fragmentos en el backend es solo la mitad del problema; la otra mitad es transportarlos hasta el navegador o la app del cliente sin perder el efecto incremental. Existen tres patrones de referencia:

- **AWS AppSync (GraphQL Subscriptions sobre WebSocket) + AWS Amplify AI Kit.** Amplify AI Kit (parte de Amplify Gen 2, paquete `@aws-amplify/ui-react-ai`) está diseñado específicamente para aplicaciones React que consumen Bedrock a través de AppSync: sus “conversation routes” usan las suscripciones GraphQL en tiempo real para entregar la respuesta token a token al cliente, mientras el resolver Lambda invocado en modo síncrono (`RequestResponse`, el valor por defecto de los resolvers Lambda de AppSync) sigue ejecutando la llamada a Bedrock en segundo plano. Es la opción de menor esfuerzo cuando la aplicación ya vive sobre la pila AppSync/Amplify, porque no exige rediseñar la arquitectura.
- **Amazon API Gateway WebSocket API + AWS Lambda.** Mantiene una conexión persistente y bidireccional entre cliente y backend; la función Lambda invoca `InvokeModelWithResponseStream` (o `ConverseStream`) y reenvía cada fragmento a la conexión activa mediante la API de administración de conexiones de API Gateway. Este patrón soporta miles de conexiones concurrentes con baja sobrecarga y es el recomendado por AWS para chat conversacional en tiempo real cuando no existe ya una capa AppSync.

- **AWS Lambda Function URLs con Response Streaming, o Server-Sent Events (SSE).** Alternativas más ligeras cuando no se necesita bidireccionalidad completa.

Hay un matiz de API Gateway que el examen evalúa con frecuencia y que conviene fijar con precisión: el “response transfer mode” **STREAM** para integraciones proxy (incluida una integración Lambda que usa *response streaming*) **solo está soportado en REST API, no en HTTP API**. Es un error común diseñar una arquitectura de streaming de bajo coste sobre una HTTP API (más barata y simple que una REST API) asumiendo que el streaming funcionará igual; si el requisito es transmitir la respuesta en tiempo real a través de una integración proxy de Lambda, la REST API es la única variante de API Gateway que lo admite hoy. Client-side polling contra una API de solicitud-respuesta única (por ejemplo, sondear cada 100 ms) no es streaming real: multiplica el número de solicitudes sin resolver el problema de fondo, porque no hay fragmentos parciales que consultar hasta que el modelo termina.

Por último, cuando la latencia del propio modelo (no la del transporte) es el cuello de botella, Bedrock ofrece **Latency Optimized Inference**: activarla mediante `performanceConfig: { "latency": "optimized" }` en la solicitud reduce el tiempo de respuesta para los modelos que la soportan sin necesidad de aprovisionar capacidad ni reentrenar el modelo. Se factura por uso, y si se agota la cuota de esta ruta optimizada, la solicitud simplemente cae a la inferencia estándar en lugar de fallar.



Mecánica de Prompt Caching en Amazon Bedrock

Caso de Estudio

Una aplicación de asistencia al cliente en tiempo real necesita mostrar sugerencias mientras el cliente todavía está hablando, con menos de 1 segundo de latencia extremo a extremo, usando únicamente servicios gestionados. La arquitectura ganadora combina tres piezas de streaming encadenadas: Amazon Transcribe en modo streaming con *partial results* habilitados (entrega fragmentos de transcripción antes de que el hablante termine la frase), reenvío inmediato de esos fragmentos a Bedrock mediante `InvokeModelWithResponseStream` (para que el modelo también genere de forma incremental), y entrega al agente humano a través de una API WebSocket de API Gateway. Los distractores fallan por razones muy distintas entre sí: usar transcripción por lotes (batch) rompe la cadena de baja latencia en el primer eslabón, sin importar cuánto se optimicen los eslabones siguientes; insertar un paso de análisis de sentimiento con Amazon Comprehend añade una parada no solicitada; y publicar resultados en Amazon SNS sustituye una conexión bidireccional de baja latencia por un mecanismo de notificación asíncrona de baja frecuencia, pensado para otro tipo de carga de trabajo. La lección general: en una cadena de streaming, la latencia final la determina el eslabón más lento o más “por lotes” de toda la cadena, no solo el paso de generación del LLM.

6.2 Prompt Caching en profundidad

Muchas aplicaciones repiten, invocación tras invocación, un bloque de contexto idéntico: una instrucción de sistema extensa, un conjunto de ejemplos *few-shot*, un documento base o un catálogo de productos. Sin caché, Bedrock tiene que volver a procesar (a tarifa completa) ese bloque en cada llamada, aunque no haya cambiado un solo carácter. **Prompt Caching** resuelve esto permitiendo marcar puntos de control (*cache checkpoints*) en el prompt: el contenido hasta ese punto se almacena en caché y, si una llamada posterior reutiliza exactamente ese mismo prefijo, Bedrock lo recupera de la caché en lugar de reprocesarlo.

Los detalles operativos importan para el examen: cada modelo define un mínimo de tokens por checkpoint (por ejemplo, Claude 3.7 Sonnet requiere al menos 1.024 tokens para poder crear un checkpoint), la caché tiene un TTL de aproximadamente cinco minutos que se renueva con cada acierto (*cache hit*), los tokens escritos en caché por primera vez pueden facturarse a una tarifa distinta de los tokens leídos desde caché, y los tokens no cacheados se cobran a la tarifa estándar de entrada. Existen además dos modalidades: **caché explícita** (el desarrollador marca checkpoints de forma deliberada) y **caché implícita**, en la que Bedrock puede reutilizar automáticamente prefijos repetidos sin necesidad de checkpoints manuales, según el modelo.

Sobre el ahorro exacto: es habitual encontrar en material de estudio la cifra “hasta 90% de ahorro en coste de tokens de entrada y hasta 85% de reducción de latencia”. Estas cifras proceden de casos de referencia publicados por AWS y son razonables como techo orientativo para prefijos muy largos con una alta tasa de aciertos de caché, pero no son una garantía fija ni universal: el ahorro real depende del modelo, de qué proporción del prompt es realmente cacheable, y de la tasa de aciertos en producción. Para el examen, lo importante no es memorizar el porcentaje exacto, sino entender el mecanismo: prompt caching ataca directamente el **coste de tokens de entrada repetidos** y el **TTFT**, no mejora la velocidad de generación de tokens de salida ni sustituye a un mecanismo de reintentos o de escalado de capacidad.

Prompt caching es también, con frecuencia, la respuesta correcta frente a alternativas que “reinventan” con infraestructura propia algo que Bedrock ya resuelve de forma nativa: montar una capa de caché en Amazon ElastiCache o en Amazon DynamoDB para las mismas consultas repetidas exige desplegar, operar y pagar

infraestructura adicional, mientras que prompt caching está integrado en la propia API de Bedrock sin componentes nuevos que gestionar.

Caso de Estudio

Un asistente de atención al cliente procesa habitualmente 50.000 consultas diarias, con picos de hasta 150.000 durante eventos promocionales, y un análisis muestra que el 40% de las consultas comparten el mismo contexto de base. La solución más rentable no es comprar Provisioned Throughput dimensionado para el pico (esa capacidad quedaría infrautilizada el resto del tiempo, ya que se factura de forma constante independientemente del uso real) ni montar una caché propia en ElastiCache o DynamoDB (duplica con infraestructura propia algo que Bedrock ya ofrece de forma nativa). La combinación ganadora es activar Latency Optimized Inference (sin coste fijo, sin aprovisionar nada, con caída elegante a inferencia estándar si se agota la cuota) junto con Prompt Caching para aprovechar exactamente ese 40% de contexto repetido. El patrón general: cuando un escenario menciona explícitamente que “una parte del tráfico comparte contexto” y pide la solución “más rentable”, casi siempre se está apuntando a Prompt Caching frente a soluciones de capacidad fija o de caché externa.

6.3 Resiliencia ante Throttling: reintentos, modo adaptativo y circuit breaker

El error HTTP 429 (Throttling) aparece cuando una aplicación supera la cuota de solicitudes por minuto (RPM) o tokens por minuto (TPM) de una región. La reacción reflexiva de subir el timeout del cliente no ataca la causa raíz: el problema no es que la respuesta tarde, es que la solicitud ni siquiera se está aceptando.

Los SDK de AWS ofrecen varios modos de reintento, y distinguirlos es un punto muy examinado:

- **Modo `legacy` /fijo**: reintentos con un retraso fijo, sin distinguir el tipo de error. No se adapta a la disponibilidad cambiante del servicio y es la opción menos recomendable.
- **Modo `standard`**: el modo por defecto recomendado por AWS; ya incorpora **backoff exponencial con jitter aleatorizado** y aplica retrasos distintos según el tipo de error (retrasos más cortos para errores transitorios de red, más largos para errores de limitación de cuota). El *jitter* — un componente aleatorio añadido al tiempo de espera — existe específicamente para evitar el problema de *thundering herd*: si todos los clientes reintentan exactamente al mismo intervalo tras un fallo masivo, generan una nueva oleada sincronizada de solicitudes que puede volver a saturar el servicio.
- **Modo `adaptive`**: añade sobre el modo estándar un limitador de tasa del lado del cliente que ralentiza automáticamente el envío de solicitudes en cuanto detecta señales de throttling del servicio, sin esperar a que se agote el presupuesto de reintentos. La documentación de referencia de los SDK de AWS lo recomienda explícitamente para “*AI workloads that call a single API operation at high volume*” — exactamente el patrón de una aplicación que invoca repetidamente `InvokeModel` o `Converse` contra Bedrock durante picos de tráfico.

Por encima del nivel de reintento individual, el **AWS Well-Architected Framework (Agentic AI Lens)** recomienda complementar el backoff adaptativo con un patrón de **circuit breaker**: un mecanismo que, cuando la tasa de errores supera un umbral predefinido, desactiva temporalmente los reintentos durante una ventana de tiempo en lugar de seguir bombardeando un servicio ya saturado, y los reactiva gradualmente cuando el servicio

se recupera. Es la diferencia entre “reintentar más inteligentemente” (adaptive) y “dejar de intentarlo durante un rato” (circuit breaker) cuando la situación es claramente insostenible.

Cuando el problema no es un pico puntual sino una cuota regional estructuralmente insuficiente, la resiliencia por reintentos deja de ser suficiente y hace falta más capacidad o más rutas: **desacoplamiento asíncrono** con una cola de Amazon SQS o un bus de Amazon EventBridge frente a los componentes consumidores, para amortiguar y aplanar picos de tráfico antes de que lleguen a Bedrock; **Cross-Region Inference Profiles**, que enrutan automáticamente el exceso de tráfico hacia otras regiones dentro del mismo perfil geográfico (`us.` , `eu.` , `apac.`) o de un perfil global, sin coste de enrutamiento adicional y facturado a tarifa on-demand estándar — la documentación de AWS lo recomienda explícitamente cuando “la disponibilidad y el throughput priman sobre el coste”; o **Provisioned Throughput**, cuando lo que se necesita es capacidad dedicada y predecible, no solo más rutas. Conviene fijar un matiz operativo poco intuitivo: los perfiles de inferencia entre regiones y el Provisioned Throughput son mecanismos distintos y, según la documentación de Bedrock, **no se combinan** en una misma solicitud — hay que elegir entre repartir tráfico on-demand entre regiones o reservar capacidad dedicada en una configuración concreta, no ambas cosas a la vez sobre el mismo perfil.

Un detalle operativo que el examen usa como trampa recurrente: reservar Provisioned Throughput no basta por sí solo. Para que una solicitud realmente utilice la capacidad reservada, hay que sustituir el `modelId` de la llamada (`InvokeModel` , `InvokeModelWithResponseStream` , `Converse` o `ConverseStream`) por el **ARN del modelo provisionado** que devuelve la API `CreateProvisionedModelThroughput` en el campo `provisionedModelArn` . Si el código sigue apuntando al identificador del modelo base (por ejemplo `anthropic.claude-v2`), todo el tráfico seguirá cayendo en la capacidad on-demand compartida — con el síntoma característico de que las métricas de CloudWatch muestran la capacidad provisionada sin uso mientras las solicitudes on-demand siguen sufriendo throttling, aunque la empresa ya esté pagando por la capacidad dedicada.

Para diagnosticar throttling con precisión, el espacio de nombres `AWS/Bedrock` de CloudWatch expone métricas específicas: `Invocations` , `InputTokenCount` , `OutputTokenCount` y, de forma crítica, `InvocationThrottles` — que cuenta específicamente las invocaciones limitadas por el sistema y que **no** se contabiliza dentro de `Invocations` ni se confunde con errores de cliente o de servidor (`InvocationClientErrors` / `InvocationServerErrors` , que miden otra cosa: fallos de la solicitud o del servicio, no limitación de cuota).

Caso de Estudio

Una aplicación educativa sobre AWS Lambda sufre throttling recurrente durante las horas punta de cada zona horaria (el tráfico se concentra por las tardes/noches locales de usuarios repartidos por el mundo), y el requisito explícito es no incurrir en un coste fijo por hora durante los periodos de baja demanda. Comprar Provisioned Throughput dimensionado para el pico global queda descartado de raíz por ese único requisito de coste, sin importar cuánto resuelva el throttling técnicamente. La solución consiste en habilitar el registro de invocaciones de Bedrock, vigilar específicamente la métrica `InvocationThrottles` (no las métricas genéricas de latencia o errores, que no distinguen throttling de otros fallos) y repartir el tráfico mediante Cross-Region Inference, que sigue facturándose a tarifa on-demand y por tanto no añade ningún coste fijo durante las horas de baja demanda de cada franja horaria. El patrón de examen: cuando un enunciado combina “picos de tráfico impredecibles o distribuidos en el tiempo” con “sin coste fijo durante baja demanda”, casi siempre descarta Provisioned Throughput y apunta a Cross-Region Inference (o, si el pico es de contenido repetido, a Prompt Caching).

6.4 Coste: eligiendo el modo de inferencia correcto

Los cuatro modos de inferencia de Bedrock (On-Demand, Provisioned Throughput, Batch Inference y Cross-Region Inference — desarrollados en profundidad en el Módulo 1) tienen perfiles de coste radicalmente distintos, y una parte importante del examen consiste en identificar cuál combina mejor con el patrón de tráfico descrito en el escenario:

- **On-Demand** es la opción por defecto para tráfico interactivo variable: se paga estrictamente por token, sin compromiso, pero está sujeto a las cuotas de TPS/TPM compartidas de la región.
- **Provisioned Throughput** convierte el coste en un cargo fijo predecible (por hora, o con descuento si se compromete a 1 o 6 meses) a cambio de capacidad garantizada; es rentable solo cuando el volumen sostenido justifica pagar por capacidad incluso en los valles de tráfico, y es la única vía para servir modelos personalizados (fine-tuned).
- **Batch Inference** ofrece el coste por token más bajo (con un descuento significativo frente a on-demand, según la página de precios de Bedrock) a cambio de renunciar a la interactividad: los trabajos se procesan de forma asíncrona leyendo y escribiendo en Amazon S3, no soportan *tool calling* ni salida estructurada multi-turno, y no son compatibles con modelos que ya tienen Provisioned Throughput asociado.
- **Cross-Region Inference** no cambia el precio por token (sigue siendo tarifa on-demand) pero cambia la disponibilidad: es la palanca correcta cuando el objetivo es throughput y continuidad de servicio, no ahorro de coste directo.

Confundir estos ejes — tratar Cross-Region como una palanca de ahorro, o Provisioned Throughput como una palanca de latencia por sí sola sin acompañarla de streaming o de Latency Optimized Inference — es la fuente más común de distractores plausibles en este bloque del examen.

Fuentes

- <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html>
- <https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/gencost03-bp03.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/latency-optimized-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prov-thru-use.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_CreateProvisionedModelThroughput.html
- <https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/capacity-limits-cost-optimization.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-runtime-metrics.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html>
- https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerateStream.html
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/response-transfer-mode.html>
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html>

- <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html>
- <https://docs.aws.amazon.com/transcribe/latest/dg/streaming-partial-results.html>
- <https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html>
- <https://docs.aws.amazon.com/wellarchitected/latest/agent-ai-lens/agentperf06-bp02.html>
- <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html> ##

MÓDULO 7: SEGURIDAD, GOBERNANZA, PRIVACIDAD Y CUMPLIMIENTO

Ninguna aplicación de IA Generativa llega a producción en una empresa real sin resolver antes cuatro preguntas: ¿quién puede invocar qué modelo?, ¿por dónde viaja el tráfico?, ¿quién puede ver qué dato? y ¿quién hizo qué y cuándo? Este módulo es, con 16 de las 97 preguntas del banco de práctica, el bloque temático más examinado después de Agentes y RAG — y no por casualidad: la inmensa mayoría de los escenarios de seguridad en el examen no piden “la solución más segura en abstracto”, sino la combinación exacta de servicios nativos de AWS que resuelve varios requisitos simultáneos (control de acceso, residencia de datos, auditoría, gobernanza de prompts) con el mínimo desarrollo a medida. Entender el propósito exacto de cada servicio — y, sobre todo, los límites de cada uno — es lo que separa la respuesta correcta de los distractores plausibles.

7.1 Control centralizado con Organizations y Service Control Policies

Cuando una organización usa AWS Organizations con todas las funciones habilitadas, puede aplicar **Service Control Policies (SCP)** que actúan como un techo de permisos sobre todas las cuentas miembro, independientemente de lo que digan las políticas de IAM locales. Para Amazon Bedrock, dos claves de condición son las que el examen espera que domines:

- **bedrock:GuardrailIdentifier** : permite denegar cualquier llamada a `InvokeModel`, `InvokeModelWithResponseStream`, `Converse` o `ConverseStream` que no incluya el identificador (y opcionalmente la versión) de un guardrail concreto. Es el mecanismo estándar para forzar que **todas** las invocaciones de Bedrock en la organización pasen por un guardrail corporativo, sin depender de que cada equipo lo configure voluntariamente en su código.
- **bedrock:modelId** (o restringir el recurso `foundation-model/*` a una lista de ARN): limita qué modelos concretos puede invocar un principal, de modo que solo los modelos “aprobados” por el equipo de gobernanza estén disponibles.

Una SCP bien diseñada combina ambas condiciones en una sola política `Deny` con un `Condition` de tipo `StringNotEquals`, de forma que cualquier llamada que no traiga el guardrail aprobado o que apunte a un modelo no autorizado se bloquea a nivel de organización, antes incluso de llegar a evaluarse la política de IAM del rol. Es importante no confundir esto con un **IAM permissions boundary**: el boundary limita el techo de permisos de un *rol* individual, pero no es el mecanismo que AWS documenta para exigir un guardrail en tiempo de invocación: añadir un boundary junto a una SCP que ya cubre ambos requisitos es una capa redundante, no una mejora de seguridad.

Las SCP también se usan para **restringir regiones** mediante `aws:RequestedRegion`, un patrón habitual cuando una empresa europea solo puede operar en `eu-central-1` y `eu-west-1` por motivos de residencia de datos. Aquí aparece uno de los matices más finos del examen: si la empresa además quiere usar **Cross-Region Inference (CRI)** — necesario, por ejemplo, para acceder a un modelo como Amazon Nova Pro que solo soporta invocación mediante *inference profile* — la SCP debe ampliarse explícitamente para permitir el **perfil de infe-**

rencia geográfico (p. ej. `eu.amazon.nova-pro-v1:0`), y los roles de IAM deben tener permiso de invocación no solo sobre ese perfil, sino sobre el ARN del modelo subyacente en **todas** las regiones de destino que ese perfil pueda usar como *fallback* dentro de la geografía — no solo en la región “principal”. Un error típico de SCP demasiado restrictiva (bloquear todo excepto `eu-central-1`) provoca exactamente el error real `"a service control policy explicitly denies the action"` cuando el perfil intenta enrutar a `eu-west-3`. La solución nunca es abrir un prefijo genérico como `eu.*` (demasiado amplio, viola el principio de mínimo privilegio) ni otorgar `AdministratorAccess` (elimina todo control); es autorizar **el perfil concreto** y sus regiones de destino conocidas.

7.2 Aislamiento de red con AWS PrivateLink

Cuando el requisito es que el tráfico entre la aplicación y Amazon Bedrock **nunca salga a Internet pública**, la respuesta es un **Interface VPC Endpoint** (basado en AWS PrivateLink) hacia los endpoints de servicio de Bedrock — típicamente `bedrock` (plano de control) y `bedrock-runtime` (invocación de modelos), y de forma análoga `bedrock-agent-runtime` cuando se usan Agents o Knowledge Bases. Con el endpoint desplegado en las subredes privadas de la VPC, las funciones Lambda o instancias que invocan Bedrock no necesitan IP pública, gateway de Internet, NAT ni VPN/Direct Connect.

Un distractor recurrente en el examen es proponer un **NAT Gateway** como solución de conectividad: un NAT Gateway traduce direcciones para tráfico **saliente hacia Internet**, por lo que usarlo para llegar a Bedrock significa que el tráfico sí sale a la red pública — justo lo contrario del requisito. Otro matiz importante: un **Gateway Endpoint** (el tipo usado para S3 y DynamoDB) no es lo mismo que un **Interface Endpoint** (el tipo que usa Bedrock); confundir ambos tipos de endpoint es un error común en las opciones incorrectas.

El aislamiento de red por sí solo no controla *quién* puede invocar el modelo dentro de esa VPC — para eso se combina con condiciones de IAM (por ejemplo, restringiendo por `aws:sourceVpc`) y, cuando se necesita control de acceso por atributo de usuario o departamento, con **IAM ABAC (Attribute-Based Access Control)**: roles con etiquetas de sesión o de principal (p. ej. `department=finance`) evaluadas en políticas que conceden permisos distintos según el atributo, en lugar de mantener un rol de IAM separado por cada departamento.

7.3 Gobernanza de datos: AWS Lake Formation frente a S3 y Glue

Cuando el escenario habla de **control de acceso a nivel de columna** sobre un data lake (por ejemplo, para que un modelo de detección de fraude solo vea columnas financieras con las columnas de PII redactadas), la respuesta correcta es casi siempre **AWS Lake Formation con LF-Tags (Lake Formation Tag-Based Access Control, LF-TBAC)**. El patrón es: se definen etiquetas clave-valor (`unidad_negocio=finanzas`, `region=eu-west-1`) y se asignan a bases de datos, tablas y, de forma crítica, a **columnas concretas** del Glue Data Catalog; después se conceden permisos sobre **expresiones de LF-Tags** en lugar de sobre cada combinación individual de principal y recurso. Esto es lo que hace que LF-TBAC **escale**: con *grants* directos (named resource method) el número de concesiones crece combinatoriamente con el número de unidades de negocio y regiones, mientras que con LF-Tags basta con añadir la etiqueta correspondiente a los nuevos recursos.

El error de diseño más repetido en los distractores es aplicar el control de gobernanza **directamente sobre objetos de S3** (bucket policies, ACL, o incluso *object tags* de S3) en lugar de sobre el catálogo de metadatos centralizado. Las etiquetas de S3 son propiedades de un objeto individual: no se integran con el modelo de

permisos de Lake Formation ni con herramientas analíticas como Athena, EMR o Redshift, y **no ofrecen ningún mecanismo de filtrado o redactado a nivel de columna** — solo Lake Formation lo hace de forma nativa. Dicho esto, S3 sí tiene un rol legítimo en gobernanza cuando el requisito es más simple: los **metadatos de sistema** de S3 (fecha de creación del objeto) sirven de forma nativa para *timestamps* de documento; los **object tags** (hasta 10 por objeto) son la herramienta correcta para clasificar por categoría o dominio; y los **metadatos definidos por el usuario** (`x-amz-meta-*`) permiten adjuntar atributos arbitrarios como el autor — sin necesidad de abrir el contenido del documento, algo valioso para que un FM entienda el contexto de un documento antes de procesarlo. La clave de examen es distinguir cuándo el requisito es “atribuir/etiquetar objetos individuales” (S3 nativo) de cuándo es “gobernar el acceso columna por columna a través de múltiples cuentas y servicios analíticos” (Lake Formation).

Para el registro y catalogación de fuentes de datos no estructuradas (transcripciones, informes, documentación) con calidad de datos verificable, el servicio correcto es **AWS Glue**: un *crawler* de Glue cataloga automáticamente el esquema en el **Glue Data Catalog** (generando metadatos auditables sin esfuerzo manual), los *ETL jobs* ejecutan la lógica de *chunking* o transformación personalizada, y **AWS Glue Data Quality** — basado en el framework open-source Deequ y su lenguaje declarativo DQDL — aplica más de 25 reglas de calidad predefinidas, detecta anomalías con ML y aísla registros problemáticos, todo en una única canalización sin servidores. Frente a esto, herramientas como Amazon SageMaker Data Wrangler (pensada para *feature engineering* tabular de ML) o Amazon Comprehend (extracción de entidades, sin reglas de calidad declarativas) exigen construir manualmente las piezas que Glue ya ofrece de forma nativa.

Caso de Estudio — LF-Tags vs. grants directos

Una aseguradora con datos financieros sensibles en su data lake necesita que un modelo de detección de fraude en Amazon Bedrock consuma solo las columnas autorizadas, con el PII de esas tablas redactado, y un registro de auditoría de cada acceso. La tentación es usar concesiones directas de Lake Formation sobre tablas concretas (*named resource method*) y añadir una capa de aplicación a medida que filtre columnas y registre accesos. Esa solución no escala: cada nueva combinación de unidad de negocio y región exige una concesión adicional, y la capa de filtrado personalizada duplica una capacidad que Lake Formation ya ofrece de forma nativa. La solución correcta es autenticar mediante roles de IAM y conceder permisos LF-TBAC sobre expresiones de LF-Tags asignadas a bases de datos, tablas y columnas, con el filtrado de columnas sensibles resuelto de forma nativa por Lake Formation y el registro de auditoría cubierto automáticamente por AWS CloudTrail (incluyendo llamadas como `GetDataAccess`).

7.4 Validación preventiva de infraestructura: AWS CloudFormation Guard

Cuando el enunciado pide “validar plantillas de CloudFormation/CDK **antes** de que los recursos se aprovisionen” o habla de “policy-as-code” en un pipeline de CI/CD, la respuesta es **AWS CloudFormation Guard** (`cfn-guard`). Es una herramienta de línea de comandos declarativa (con su propio DSL de reglas) que evalúa una plantilla contra un conjunto de políticas — por ejemplo, “todo bucket S3 debe tener cifrado KMS” o “ningún endpoint puede ser público” — y falla el pipeline **antes del despliegue** si detecta una violación, dando retroalimentación temprana al desarrollador.

El matiz que el examen explota repetidamente es la diferencia entre **prevención y detección**: si una opción describe ejecutar `cfn-guard` **después** de que los recursos ya se desplegaron, esa opción es incorrecta — el valor

de `cfn-guard` está en el *gate* de CI/CD, no en una auditoría posterior, porque para entonces el recurso no conforme ya existe. Ese rol reactivo (evaluar el cumplimiento de configuración de recursos ya desplegados) es el que corresponde a **AWS Config**, no a `cfn-guard`. Del mismo modo, **Amazon Macie** (clasificación de datos sensibles en S3) y **AWS Service Catalog** (portfolios de productos versionados, pero con despliegue manual desde consola) resuelven problemas de gobernanza distintos: ninguno sustituye a un *gate* automatizado de políticas como código integrado en el pipeline. El patrón de referencia que documenta el AWS Well-Architected Generative AI Lens para múltiples unidades de negocio es: plantillas de CloudFormation estandarizadas y versionadas en un repositorio central + pipeline de CI/CD que integra `cfn-guard` como paso de validación previo al despliegue.

7.5 Gobernanza de Amazon Q Business y Amazon Q Developer

Dos patrones de Amazon Q aparecen de forma recurrente y con una redacción muy parecida a propósito, para poner a prueba si conoces el mecanismo exacto:

- **Control de acceso en el conector S3 de Amazon Q Business:** se configura mediante un **único archivo JSON** (comúnmente `acl.json`, referenciado por el parámetro `aclConfigurationFilePath`) colocado en el bucket, que contiene un array de entradas: cada elemento define un `keyPrefix` (el prefijo S3 de un departamento) junto con una lista de `aclEntries` (`Name`, `Type`: `USER` / `GROUP`, `Access`: `ALLOW` / `DENY`). **No** se crean archivos `acl.json` independientes por carpeta de departamento — eso multiplica el mantenimiento sin beneficio. Detalle importante para producción: cualquier prefijo que **no** aparezca en el archivo queda accesible para **todos** los usuarios, así que el archivo debe ser exhaustivo.
- **Amazon Q Business Data Accessor:** el patrón correcto cuando un ISV (proveedor de software) necesita enriquecer prompts con datos empresariales de **múltiples clientes externos**, sin desplegar infraestructura en el entorno de cada cliente. Cada cliente configura su propio índice de Q Business sobre sus datos internos y designa al ISV como *data accessor* verificado; el ISV consulta después la API `SearchRelevantContent`, que respeta los controles de acceso del cliente (solo devuelve lo que el usuario final podría ver). Esto es preferible a que cada cliente construya su propia Knowledge Base de Bedrock con consultas cross-account, o a desplegar servidores MCP en tiempo real en cada entorno cliente: ambas alternativas multiplican la infraestructura a mantener por cliente, algo explícitamente descartado cuando el requisito es “mínima complejidad arquitectónica” a escala de millones de clientes.
- **Amazon Q Developer Customizations** frente a `.amazonq/rules`: las *customizations* son personalizaciones a nivel de cuenta/organización, configuradas una sola vez por un administrador (conectando repositorios internos vía S3 o AWS CodeConnections) para que las sugerencias *in-line* de código respeten las bibliotecas, algoritmos y estilos propios de la empresa, sin tocar la configuración de cada proyecto. La carpeta `.amazonq/rules` en la raíz de un proyecto, en cambio, almacena *reglas de proyecto* en Markdown que Amazon Q usa como contexto de chat **para ese proyecto concreto** — útil, pero es exactamente el tipo de configuración “a nivel de proyecto” que una customization centralizada evita cuando el requisito explícito es no modificar cada proyecto individualmente.

7.6 Gestión del ciclo de vida de prompts: Amazon Bedrock Prompt Management

Cuando el requisito de gobernanza se refiere a **plantillas de prompts** (no a datos ni a infraestructura), el servicio nativo es **Amazon Bedrock Prompt Management**: permite crear prompts reutilizables, parametrizarlos con variables (`{{variable}}`), y crear **versiones inmutables** de cada prompt con capacidad de comparar versiones y desplegar una concreta en producción. El control de **quién puede crear o publicar** una versión se resuelve con políticas de IAM basadas en identidad sobre las operaciones de la API (`CreatePrompt` , `CreatePromptVersion` , `UpdatePrompt`), y el rastro de auditoría lo proporciona **AWS CloudTrail**, que registra automáticamente esas llamadas como eventos de gestión. Vale la pena memorizar un matiz honesto: Prompt Management **no** incluye una funcionalidad nativa de notificaciones push de “revisión pendiente” — ese aspecto se aproxima mediante permisos de IAM (solo un rol “aprobador” puede publicar una versión), no mediante una alerta automática del servicio. Reconstruir todo esto desde cero con Step Functions, S3 y EventBridge sí cubriría las notificaciones de forma más directa, pero renuncia al versionado inmutable y a la integración nativa con el resto de Bedrock que ya ofrece Prompt Management sin desarrollo adicional.

7.7 Auditoría, cifrado y visibilidad operativa

El requisito “quién accedió a qué y cuándo” combina siempre tres piezas: **control de acceso** (IAM/ABAC + condiciones de guardrail/modelo), **cifrado** (SSE-KMS en S3 y en los buckets de logging) y **registro de eventos**. Aquí el detalle que distingue la respuesta correcta de un distractor plausible es la **granularidad** del registro:

- **AWS CloudTrail** con **eventos de gestión y de datos** habilitados sobre objetos de S3 y sobre el uso de claves de KMS, enviado opcionalmente a **AWS CloudTrail Lake** para consultas SQL retenidas, es el mecanismo que documenta de forma completa quién invocó qué API, cuándo y desde dónde.
- **Amazon CloudWatch Logs** registra métricas y logs de aplicación, pero no sustituye a CloudTrail como registro de auditoría de llamadas a la API con identidad del llamador.
- Los **trace events de Bedrock Agents** (ver Módulo 5) auditan el razonamiento interno de un agente, pero no el acceso a nivel de archivo a un almacén de datos como S3.
- El **S3 server access logging** añade el detalle de interacción a nivel de archivo individual que CloudTrail por sí solo, en su configuración básica, puede no capturar con el mismo nivel de detalle por objeto.
- **AWS CloudTrail Insights** detecta patrones anómalos de volumen de llamadas, útil para alertar, pero no es un registro exhaustivo de cada acceso individual — hace falta combinarlo con los eventos estándar de gestión y de datos.

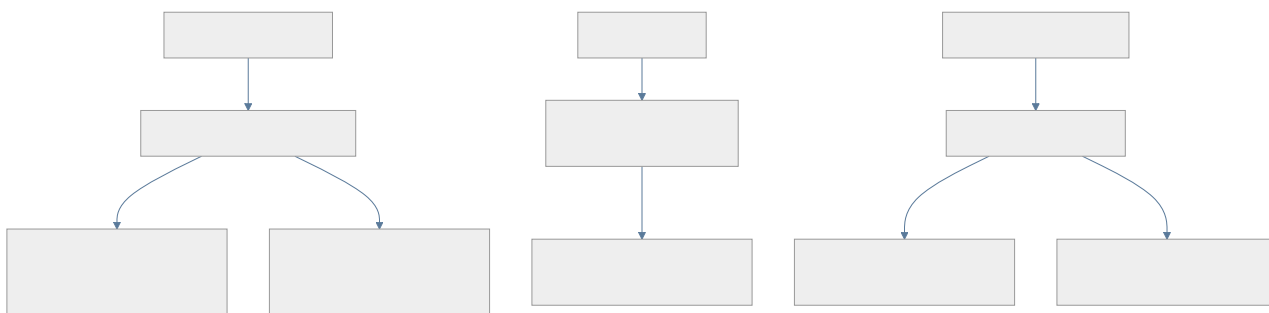
Un escenario típico exige combinar varias de estas piezas: VPC endpoints para I/O seguro del modelo, ABAC + `bedrock:GuardrailIdentifier / modelId` para controlar quién invoca qué, CloudTrail (gestión + datos) y S3 server access logging para la auditoría fina, y SSE-KMS para el cifrado en reposo. Una opción que solo cubre la parte de red y guardrails, pero omite el control de acceso granular o el registro a nivel de archivo, resuelve la seguridad de la conexión pero no la trazabilidad completa que exige el enunciado.

Caso de Estudio — Residencia de datos y Cross-Region Inference geográfico

Una empresa europea, con SCPs que restringen sus cuentas a `eu-north-1` y `eu-west-1`, selecciona un modelo alojado en `eu-central-1` y `eu-west-3`, y necesita que las peticiones de sus empleados lleguen a ese modelo de forma privada y sin salir de Europa. Desplegar el modelo en EC2 o en SageMaker no tiene sentido: Bedrock es un servicio gestionado que no distribuye pesos de modelo para autoalojamiento, y renunciar a Bedrock tras haberlo evaluado duplicaría trabajo. La solución correcta combina tres piezas: un **inference profile geográfico** (`eu.*`) para que Bedrock enrute automáticamente entre regiones europeas sin cruzar el continente; un **VPC endpoint de interfaz** para que el tráfico entre la VPC corporativa y Bedrock no toque Internet pública; y una **ampliación explícita de las SCP** para autorizar las regiones de destino reales del perfil (`eu-central-1`, `eu-west-3`), además de las ya permitidas. Omitir la ampliación de la SCP es el error más común: el perfil de inferencia no puede saltarse una denegación explícita a nivel de organización.

Caso de Estudio — Bloquear un tema, no solo enmascararlo

Una organización con SCPs a nivel de AWS Organizations quiere impedir que información propietaria y ciertos temas lleguen a los prompts de Bedrock, y que los empleados solo usen modelos aprobados. La combinación ganadora es una SCP que restringe los modelos permitidos y exige un `bedrock:GuardrailIdentifier` aprobado en cada llamada, más un guardrail desplegado de forma consistente en todas las cuentas mediante **CloudFormation StackSets**, configurado con una **política de bloqueo (block)** sobre los temas denegados. El matiz que separa esto de un distractor: una política de **enmascaramiento (mask)** en los filtros de información sensible sustituye el dato por una etiqueta pero deja que la solicitud siga procesándose — sirve para PII que debe seguir fluyendo de forma anonimizada, no para **impedir** que un tema o un dato propietario llegue al modelo. Cuando el verbo del enunciado es “prevenir” o “bloquear”, la política debe ser `block`, no `mask`.



Capas de seguridad y gobernanza para cargas de trabajo GenAI en AWS: control organizativo (SCP), aislamiento de red (PrivateLink), gobernanza de datos (Lake Formation) y validación preventiva de IaC (cfn-guard).

Fuentes

- <https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-permissions-id.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/geographic-cross-region-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/inference-profiles-support.html>

- <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-sensitive-filters.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html>
- <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-cross-account.html>
- <https://docs.aws.amazon.com/lake-formation/latest/dg/tag-based-access-control.html>
- <https://docs.aws.amazon.com/lake-formation/latest/dg/cross-account-TBAC.html>
- <https://docs.aws.amazon.com/lake-formation/latest/dg/logging-using-cloudtrail.html>
- <https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/best-practices.html>
- <https://docs.aws.amazon.com/cfn-guard/latest/ug/what-is-guard.html>
- <https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-ai-lens.html>
- <https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/s3-user-management.html>
- <https://docs.aws.amazon.com/amazonq/latest/qbusiness-ug/isv.html>
- <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/context-project-rules.html>
- <https://docs.aws.amazon.com/prescriptive-guidance/latest/best-practices-code-generation/advanced-capabilities.html>
- <https://docs.aws.amazon.com/glue/latest/dg/glue-data-quality.html>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingMetadata.html>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-tagging.html>
- <https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html> ## MÓDULO 8: MATRIZ DE DECISIÓN RÁPIDA Y CHEAT SHEET DEL EXAMEN

Tabla de Decisión Rápida: “¿Qué Servicio / Técnica Elegir?”

Si el escenario pide...	La Solución Correcta en AWS es...	Distractor Común (Respuesta Incorrecta)
Enrutar dinámicamente preguntas entre varios agentes especializados	Multi-Agent Collaboration con Supervisor Agent o Bedrock Flows	Crear un único megamodelo con un prompt gigante no estructurado
Ejecutar llamadas a APIs locales/ on-premises sin exponer credenciales a AWS	Bedrock Agents con Return of Control (RoC)	Crear funciones Lambda con VPC Peering complejo o almacenar claves privadas en S3
Reducir costes un 50% en procesamiento masivo de datos no interactivo	Bedrock Batch Inference leyendo/escribiendo en S3	Provisioned Throughput con sobredimensionamiento de MUs
Evitar que el LLM invente datos en un sistema RAG	Bedrock Guardrails con Contextual Grounding (Grounding Score alto)	Reducir la temperatura a 0 en el prompt (reduce creatividad pero no garantiza grounding)

Si el escenario pide...	La Solución Correcta en AWS es...	Distractor Común (Respuesta Incorrecta)
Cumplir regulaciones de residencia de datos en Europa usando Cross-Region Inference	Inference Profile con prefijo geográfico eu. (ej. <code>eu.anthropic.claude-3-5...</code>)	Cross-Region sin restricción o enrutamiento manual con API Gateway entre regiones
Unificar llamadas a modelos de diferentes proveedores con el menor esfuerzo	Amazon Bedrock Converse / ConverseStream API	Escribir adaptadores personalizados con <code>InvokeModel</code> para cada proveedor
Búsqueda semántica sobre documentos técnicos con códigos de error y números de serie	Hybrid Search (Vector Search + Búsqueda Léxica BM25)	Búsqueda puramente vectorial con embeddings estándar
Búsqueda RAG sobre entidades con dependencias y relaciones complejas	Amazon Neptune Analytics (Graph RAG)	Amazon DynamoDB o Amazon OpenSearch estándar sin grafo
Validar plantillas IaC de GenAI antes del despliegue en CI/CD	AWS CloudFormation Guard (cfn-guard)	AWS Config (es reactivo post-despliegue, no preventivo en CI/CD)
Evaluar sesgo y equidad entre grupos demográficos en tiempo real	Amazon SageMaker Clarify con métricas en CloudWatch	Amazon CloudWatch Logs Insights con expresiones regulares
Configurar permisos de acceso a archivos S3 en Amazon Q Business	Un único archivo JSON referenciado por <code>aclConfigurationFilePath</code>	Crear archivos <code>acl.json</code> independientes dentro de cada subcarpeta
Reducir latencia y coste en prompts repetitivos muy extensos	Prompt Caching de Amazon Bedrock / Anthropic	Fine-Tuning del modelo (mucho más costoso y complejo)

Los 10 “Gotchas” y Trampas Frecuentes del Examen

- Retrieve vs RetrieveAndGenerate** : Si la aplicación solo necesita los fragmentos para su propio frontend o pipeline personalizada, usa `Retrieve` . Si quieres que Bedrock invoque automáticamente el modelo y devuelva el texto final con citas, usa `RetrieveAndGenerate` .
- Guardrails MASK vs BLOCK**: Si el requisito es que la conversación *continúe* pero sin exponer datos personales (PII), la acción debe ser **MASK**, no **BLOCK**.
- Provisioned Throughput no reduce la latencia por sí solo**: Reservar MUs garantiza *capacidad y rendimiento* (evita 429), pero para optimizar latencia de respuesta se usa streaming, modelos más ligeros o `performanceConfig: { "latency": "optimized" }` .
- Lake Formation vs S3 Object Tags**: Para gobernanza unificada, control por columnas y anonimización de PII, se usa **AWS Lake Formation (LF-Tags)** sobre el Glue Data Catalog, NO etiquetas directas en objetos de S3.

5. **Bedrock Trace Events:** Los fallos en la orquestación del agente se auditan en el `OrchestrationTrace` / `FailureTrace` , no en las métricas estándar de CloudWatch.
6. **Automatic vs Human Model Evaluation:** Criterios como *exactitud de clasificación*, *toxicidad* y *ROUGE/ BLEU* se evalúan con **Automatic Evaluation**. Criterios subjetivos como *estilo*, *adecuación de marca* y *tono* requieren **Human Evaluation (SageMaker Ground Truth)**.
7. **Cross-Region Inference Profiles:** No transfieren datos de entrenamiento; solo balancean dinámicamente las solicitudes de inferencia en tiempo de ejecución respetando la zona geográfica configurada.
8. **OpenSearch Serverless Vector Engine:** No requiere gestionar clústeres, nodos ni sharding; escala automáticamente mediante *OpenSearch Compute Units (OCUs)*.
9. **Hierarchical Chunking (Parent-Child):** Siempre es la respuesta cuando la búsqueda debe ser muy precisa en términos específicos pero la respuesta del LLM requiere el contexto amplio de la sección completa.
10. **cfn-guard en CI/CD:** Siempre que el enunciado pida “validación preventiva antes de desplegar recursos en la cuenta”, la herramienta adecuada es `cfn-guard` .